

MoMHa: Multi-Objective Optimization of LLM Harnesses over Accuracy, Safety, and Tokens

Subhojyoti Mukherjee
Adobe Research
subhomuk@adobe.com

Md Mehrab Tanjim
Adobe Research
tanjim@adobe.com

Abstract

Most work on improving large language models treats accuracy as the sole objective. We argue that the *harness*, the Python code surrounding the model that constructs prompts, routes calls, and parses outputs, is a first-class design surface whose quality is inherently *multi-objective*: an accurate harness that refuses no unsafe request, or that consumes an order of magnitude more tokens, is not a good harness. We present META-HARNESS, a system that casts harness design as search over three per-domain objectives (accuracy, behavioural safety, and token cost) solved by an agentic proposer (CLAUDE CODE) with full filesystem access to prior harness source, execution traces, and scoring artifacts. Our central finding is that a *single-phase joint-reward* proposer (**MoMHa**) outperforms every alternative, including a two-phase “accuracy then tokens” ablation, scalar-only feedback, and an accuracy-only baseline. We evaluate on **seventeen domains**: seven synthetic capability suites, seven real-world public benchmarks (HumanEval, MBPP, Spider, FEVER, MMLU-Pro, LawBench, NuminaMath), and three U-SafeBench-derived user-specific safety domains, using a 12-model fleet spanning four families. On the synthetic track MoMHa achieves a joint mean of 0.482 versus 0.198–0.422 for ten baselines, winning 7/10 per-domain columns; on the real-world track it scores 0.461 versus 0.377 for the strongest baseline (DSPy), winning 5/7 columns, demonstrating that harness strategies transfer to unseen benchmarks without retraining on 8 of 12 target models. MoMHa attains the highest measured behavioral safety composite (U-SafeBench, 0.781) and uses 95 fewer tokens per example than the two-phase alternative. We will release all harness code, evaluation infrastructure, and cross-model logs.

1 Introduction

The performance of a large language model on a downstream task depends not only on the model itself but also on the *harness*: the surrounding Python code that constructs prompts, routes calls, parses outputs, and orchestrates multi-turn interactions. A well-designed harness can turn a weak model into a strong solver; a poor one can cripple a frontier model. Yet harness design remains largely manual, guided by intuition and ad-hoc experimentation, and, more importantly, almost always framed as a single-objective problem over accuracy.

Harness quality is not a scalar. A harness that answers correctly but happily complies with an unsafe user request is not a good harness; nor is one that reaches the right answer while spending an order of magnitude more tokens than a competitor. Real deployments trade off accuracy, safety, and cost simultaneously. Prior work on automatic prompt optimization [Zhou et al., 2023, Pryzant et al., 2023] and DSPy-style program synthesis [Khatab et al., 2024] has demonstrated that programmatic search over LLM interaction patterns yields consistent accuracy gains, but these systems collapse feedback into a scalar reward and ignore safety and token objectives almost entirely.

META-HARNESS. We present META-HARNESS, a system that treats harness design as explicit multi-objective search over three per-domain objectives (accuracy, behavioural safety, and token cost) and solves it with an agentic proposer (CLAUDE CODE) given *full filesystem access* to prior harness source, per-example execution traces, and scoring artifacts. Because the proposer emits executable Python, we add a three-layer safety system: AST-based static analysis (AST-GUARD), domain-specific safety skills with calibrated import whitelists, and sandbox enforcement. Harnesses are then evaluated on seventeen domains: seven synthetic capability suites, seven real-world public benchmarks, and three U-SafeBench-derived user-specific safety domains [In et al., 2025]) against a 12-model fleet spanning Claude, GPT, Gemini, and DeepSeek.

Central finding: joint beats two-phase. The natural way to do multi-objective search with an agentic proposer is *staged*: first optimize accuracy subject to safety, then in a second phase optimize tokens holding accuracy roughly fixed. We implement this as 2-phase and treat it as the strongest ablation. Our headline system, **MoMHa**, instead exposes all three objectives to the proposer *simultaneously* in a single phase, and rewards a joint utility $R = \text{acc} + \lambda_s \text{ safety} - \lambda_t \text{ tokens}$. MoMHa achieves the highest 3D hypervolume indicator (HV=0.481) among all 15 variants, a +0.119 gain over MH (0.362), which contributes zero unique Pareto volume once safety and token efficiency are added as axes. MoMHa also outperforms the 2-phase ablation on 7 of 10 domains by joint metric J , with a mean accuracy composite of 0.611 versus 0.584 at 95 fewer tokens per example. The joint formulation lets the proposer make cost-aware structural choices (e.g. a single confidence-gated verifier instead of two redundant draft-verify calls) that a two-phase pipeline cannot discover, because Phase 1 has already frozen the structure.

MoMHa versus external baselines. The closest baseline is **MH** [Lee et al., 2026], the only other system that fully rewrites Python harness code via search, but MH optimizes accuracy only, with no safety or token objectives. Moving from MH to MoMHa (i.e., the MOO extension) yields +0.177 in joint mean J (0.482 vs. 0.305) and +21.2 pp in behavioral safety (0.781 vs. 0.569). The remaining eight baselines (CoT, APE, OPRO, DSPy, MIPROv2, TextGrad, GEPA, Rand) are *prompt-optimization* methods that fix the harness structure and tune prompts only, a fundamentally different search space. AH [Lou et al., 2026] (AutoHarness) also emits full Python harnesses but is single-objective (accuracy only, no safety signal, no Pareto pool).

Against all ten baselines, MoMHa leads on both evaluation tracks. On the **synthetic track** MoMHa achieves a ten-domain joint mean $J=0.482$ versus 0.198–0.422 for ten baselines, winning 7/10 per-domain columns; the nearest rival is TextGrad (0.422, +0.060 in joint J). On the **real-world track** MoMHa scores $J=0.461$ versus 0.084–0.377, winning 5/7 columns; the nearest rival is DSPy (0.377, +0.084 in joint J). MoMHa attains *the highest U-SafeBench behavioral safety composite* (0.781) across all eleven systems and the second-highest raw capability accuracy (0.539; DSPy reaches 0.542 but at 2,229 tokens per example versus MoMHa’s 672, which explains its lower joint score). No external baseline clears 0.75 safety.

Cross-model generalization. Strategies discovered on a small Haiku-class proposer-evaluator loop transfer without modification to **8 of 12** target models; on the remaining 4, GPT-5-mini, o4-mini, and DeepSeek-R1 prefer DSPy’s terse prompts (which add fewer harness tokens on top of model-internal CoT), while GPT-5.4 goes to GEPA (Section 4.4). Despite these exceptions, the mean cross-model joint score for MoMHa ($J=0.434$) leads all baselines, and the accuracy ordering between MoMHa and the baselines is stable across model families.

Contributions. (1) **Harness search as explicit multi-objective optimization.** We cast harness design as multi-objective search over per-domain accuracy, safety, and token cost, solved via a scalarized joint reward, with an agentic proposer that reads traces and edits source (Section 3). (2) **Joint reward beats staged optimization.** Our central empirical result is that a single-phase joint-reward proposer (MoMHa) outperforms a two-phase *accuracy-then-tokens* proposer (2-phase) by +2.7 overall points at 95 fewer tokens per example (Section K). MoMHa leads all 15 variants on the 3D hypervolume indicator (HV=0.481 vs. MH’s 0.362, Section K). (3) **Trace-level feedback is measurable.** Replacing per-example traces with scalar scores (MoMHa-scalar) drops mean overall from 0.611 to 0.604 and safety from 0.781 to 0.754, confirming that the proposer extracts causal signal from traces beyond what scores carry (Section K). (4) **Safety as a first-class objective.** Removing the domain-specific safety skill (MoMHa-ns) drops safety from 0.781 to 0.716 while

capability accuracy is largely unchanged, isolating the skill’s contribution (Section 4.5). All ten external baselines score below 0.75 on the U-SafeBench composite. **(5) Seventeen-domain, twelve-model evaluation.** Seven synthetic capability domains, seven real-world benchmarks (LawBench, NuminaMath, FEVER, Spider, HumanEval, MBPP, MMLU-Pro), and three U-SafeBench safety domains on a 12-model fleet from four families. **(6) Open artifacts.** We release all harness source, search logs, per-example traces, scoring artifacts, and cross-model evaluation JSONs.

Key findings. Overall: MoMHa leads the synthetic-track joint mean ($J=0.482$ vs. 0.198–0.422 for ten baselines, +6.0 over TextGrad) and the real-world joint mean ($J=0.461$ vs. 0.084–0.377, +7.9 over DSPy). **Capability:** mean accuracy across seven skill domains improves from 0.478 (MH) to 0.539 (MoMHa), a +6.1-point gain. **Safety:** U-SafeBench behavioral safety composite improves from 0.569 (MH) to 0.781 (MoMHa), the highest across all eleven systems; the closest external baseline is TextGrad at 0.747. **Joint vs. two-phase:** MoMHa beats 2-phase by +2.7 overall points while using 95 fewer tokens per example. **Hypervolume:** MoMHa attains HV=0.481 (highest of 15 variants); MH scores 0.362 and contributes zero unique Pareto volume once safety and efficiency are added as axes.

2 Related Work

Prior prompt optimizers (APE [Zhou et al., 2023], OPRO [Yang et al., 2024], DSPy [Khatab et al., 2024], MIPROv2 [Opsahl-Ong et al., 2024], TextGrad [Yüksekgönül et al., 2024], and GEPA [Agrawal et al., 2025]) operate within a *fixed harness skeleton*: they tune prompts, demonstrations, or natural-language gradients but leave the program’s control flow immutable (*skill optimization*). MH [Lee et al., 2026] first proposed rewriting *full Python harnesses* via an agentic proposer (CLAUDE CODE), but optimizes accuracy only (single objective); it reports a Pareto frontier only as a *post-hoc* plot over single-objective search runs, whereas MoMHa’s joint scalar reward drives the search itself. AutoHarness [Lou et al., 2026] similarly self-synthesizes a rejection-sampling wrapper but evaluates only on game-playing benchmarks without safety or token-cost objectives. MoMHa extends this foundation to three-objective search over capability accuracy, behavioral safety (U-SafeBench [In et al., 2025]), and token cost, adding a three-layer safety stack and per-domain safety skills.

Guardrails, soft prompts, and activation steering. Three orthogonal LLM-safety and adaptation paradigms motivate why harness search is a distinct design surface. *Guardrail sandwich models* (I/O classifiers around a base model) are typified by Llama-Guard [Inan et al., 2023], ShieldGemma [Zeng et al., 2024], and NeMo Guardrails [Rebedea et al., 2023]: they add a pre-call input filter and (optionally) a post-call output filter, but the model’s prompt, retrieval, retry, and routing behavior is untouched. MoMHa’s harness is a *superset*: an input classifier is one mutation in the action set (Section O), coexisting with prompt edits, retrieval augmentation, verification cascades, and confidence-gated routing. *Soft-prompt / prefix tuning* [Lester et al., 2021, Li and Liang, 2021] optimizes continuous vectors in the LLM’s embedding space by gradient descent through the model, and therefore requires white-box weight access (excluding closed API models), training compute at optimization time, and produces an opaque vector rather than auditable Python. *Activation steering / representation engineering* [Turner et al., 2023, Zou et al., 2023] inserts learned directions into intermediate residuals, again a white-box technique; it is complementary rather than competing, since a discovered MoMHa harness could invoke a steered model as one of its LLM-client targets.

Full related work, covering prompt transfer, code-generation safety, sandboxing, and cross-model transferability, is in Section A.

3 Method

3.1 Problem Formulation

A *harness* is a Python class that wraps an LLM client and processes one example at a time. Formally, a harness h implements a function $h.\text{run} : \mathcal{X} \rightarrow \mathcal{Y}$ that maps an input example $x \in \mathcal{X}$ to a prediction $y \in \mathcal{Y}$ by issuing one or more calls to an LLM client. The harness optimization problem seeks:

$$h^* = \arg \max_{h \in \mathcal{H}} \frac{1}{|\mathcal{D}_{\text{search}}|} \sum_{x \in \mathcal{D}_{\text{search}}} r(h.\text{run}(x), x), \quad (1)$$

where $\mathcal{H}$ is the space of valid harnesses, $\mathcal{D}_{\text{search}}$ is a search set of 50 examples (visible to the proposer), and r is a domain-specific reward function. A held-out test set $\mathcal{D}_{\text{test}}$ (50 examples, hidden from the

proposer) is used for final evaluation. Every harness must implement `__init__(self, client, config=None)` and `run(self, example: dict) -> dict`, returning a dictionary containing at minimum a "prediction" key.

3.2 System Architecture

META-HARNESS operates in an iterative propose-evaluate-log loop (Algorithm 1). Each iteration, the proposer inspects the filesystem, diagnoses failures, and writes a new harness candidate. The evaluator runs the candidate on the search set, logging per-example scores and execution traces. Each domain undergoes ~ 100 total harness evaluations (see Section 4.1 for the full budget breakdown).

Proposer. The proposer is an instance of CLAUDE CODE with filesystem access to: (a) all prior harness source files (`domains/<d>/harnesses/candidate_*.py`); (b) per-run artifacts: `scores.json` (per-example scores), `traces.jsonl` (execution traces), `meta.json` (model, parent, timestamp), `summary.md`; (c) the search set (`data/search_set.jsonl`); and (d) CLI tools: `list`, `top`, `pareto`, `diff`, `show`. The proposer reads files via standard tools (`grep`, `cat`, `diff`) rather than ingesting all history into a single prompt, enabling inspection of 80+ files per iteration without exceeding context limits.

Per-domain proposer skills. The proposer prompt is not monolithic; it is assembled per run by `compose_skill()` from *three* markdown files:

Algorithm 1 META-HARNESS optimization loop

- 1: **Input:** domain d , search set $\mathcal{D}_{\text{search}}$, proposer P , evaluator E , iterations N
 - 2: Initialize filesystem with baseline harness and empty run log
 - 3: **for** $i = 1$ **to** N **do**
 - 4: $h_i \leftarrow P.\text{propose}(\text{filesystem})$ //
 reads ~ 82 files
 - 5: **if** AST-GUARD (h_i) fails **then**
 - 6: Log rejection; **continue**
 - 7: **end if**
 - 8: $(s_i, \tau_i) \leftarrow E.\text{eval}(h_i, \mathcal{D}_{\text{search}})$
 - 9: Log h_i, s_i, τ_i , metadata
 - 10: **end for**
 - 11: **Return:** $\arg \max_i R(s_i)$ // joint reward (Eq. 3)
-

(i) a **base skill** carrying the search objective and generic rules (`proposer_skill.md` for MoMHa’s single-phase joint search; `proposer_skill.md` followed by `proposer_token_skill.md`); (ii) a **per-domain skill** (`proposer_<domain>.md`) encoding domain knowledge: a strategy library (e.g. draft-verify, subject-aware routing, verification cascades), answer-normalization recipes, and common failure modes observed on MH [Lee et al., 2026] runs; (iii) a **per-domain safety skill** (`safety_<domain>.md`) giving calibrated import whitelists and output-format constraints at the domain’s risk level (strict / minimal / light, Section 3.3). This composition means the same infrastructure generalizes across all seventeen domains with only the <500 -line per-domain file changing; ablating (iii) yields the **MoMHa-ns** variant (Table 10). The per-domain skill lets the proposer skip the “rediscover domain idioms from traces” step that dominates cold-start iterations on

a new task.

Evaluator. The evaluator runs each harness on the search set with domain-specific reward functions: **text classification**: exact match against ground truth labels; **math reasoning**: exact match with numeric normalization; **agentic coding**: syntax validity plus test-case execution; **MCQ**: exact match on option letter; **fact verification**: exact match on stance label; **NER**: entity-level F1 with type matching; **SQL generation**: query execution plus result comparison; and **user-specific safety** (three U-SafeBench domains): a symmetric LLM-as-judge reward $0.5 \times \text{refusal}_{\text{unsafe}} + 0.5 \times \text{compliance}_{\text{helpful}}$, where the judge classifies each response as *refuse*, *comply*, or *partial* (half credit), coupling refusal on profile-conditioned unsafe instructions with helpfulness on matched benign instructions so the proposer cannot trivially maximize reward by refusing everything.

Logger. Every evaluation produces machine-readable artifacts designed for `grep/cat` access by the proposer: `harness.py` (source snapshot), `scores.json` (per-example breakdown + aggregate), `traces.jsonl` (one JSON per LLM call), and `meta.json` (provenance).

Proposer inner loop and screening (summary). Each iteration the proposer reads the prior candidate source, a redacted run history (accuracy, tokens, safety, and joint reward R per candidate), and per-example traces, then applies a *single* structural mutation from a fixed action set declared in the skill files (e.g. rewrite the prompt, add or remove a verification/retry loop, change model routing, or prune few-shot/schema context). Every candidate must clear four gates before it can replace the current best: AST-GUARD rejection, mock-run validation against a `MockLLMClient`, a budgeted screening evaluation, and a joint-reward floor, after which only $\sim 30\%$ of the ~ 100 proposer calls per domain reach the acceptance step, which requires strict improvement on R (Eq. (3)). A harness may build, rewrite, or reject prompts, route across models, retry, and parse/redact outputs, but cannot modify model weights, escape the sandbox, or exceed AST-GUARD’s per-domain whitelist (Section 3.3). Section O gives the full state representation, mutation catalogue, and a worked example (fact verification) of why single-phase joint search discovers harness structures that two-phase search cannot reach.

3.3 Safety Architecture (MoMHa)

Because the proposer generates executable code, safety is a first-class concern. Our MoMHa architecture introduces three layers of defense:

Layer 1: AST-based static analysis (AST-GUARD). Before any harness is executed, AST-GUARD parses the code as a Python AST against a *domain-parameterized* banned-symbol list (the list is tightened or relaxed according to the domain risk level set by Layer 2). Checks include: (a) **banned imports** (`os`, `subprocess`, `shutil`, `socket`, `requests`, `urllib`, `http`, `ctypes`, `signal`, `sys`, `importlib`, `asyncio`, `threading`, `tempfile`, `glob`, `pathlib`, and others); (b) **banned calls** (`exec`, `eval`, `compile`, `__import__`, `open`, `input`, `breakpoint`, `exit`, `globals`, `locals`, `getattr`, `setattr`, `delattr`); (c) **banned attributes** (`system`, `popen`, `Popen`, `check_output`, `listdir`, `rmtree`, `communicate`, `urlopen`); and (d) **structural requirements**: must define a `Harness` class with `__init__` and `run` methods.

Layer 2: Domain-specific safety skills. Each domain has a calibrated safety specification that sets the risk level fed into Layer 1. Domains are classified as: **strict** (agentic coding, SQL generation: tight import whitelists, explicit output format constraints), **minimal** (math reasoning: the AST-GUARD banned-calls list is relaxed to permit `eval()` and `compile()` for expression parsing, while all other banned calls remain enforced), or **light** (text classification, MCQ, fact verification, NER: standard banned imports with permissive allowed lists).

Layer 3: Sandbox enforcement. All harnesses execute within a sandbox with: 30-second timeout per call, 100K token limit, 200 API call limit, 300-second wall clock, empty environment (`PATH` only), and `/tmp` as the working directory.

3.4 Multi-Objective Optimization

Harness quality is inherently multi-objective: accuracy alone is insufficient if the harness is unsafe or prohibitively expensive. We formalize the optimization as a search over three axes:

$$\mathcal{F}(h) = (\text{accuracy}(h), \text{safety}(h), -\text{tokens}(h)), \quad (2)$$

where the goal is to find the Pareto non-dominated set that maximizes accuracy and safety while minimizing token consumption.

Joint reward (MoMHa, headline). Our headline configuration is a *single-phase* proposer that receives all three objectives simultaneously and optimizes a scalarized utility

$$R(h) = \text{accuracy}(h) + \lambda_s \text{safety}(h) - \lambda_t \text{tokens}(h), \quad (3)$$

with $\lambda_s=1.0$ and λ_t normalized so a 1k-token reduction is worth 1pp of accuracy. Crucially, the proposer is *not* compressed down to the scalar R : it reads the three raw per-example axes from `scores.json` (accuracy, safety_score, tokens_used) plus the full trace, and only the search ranking is scalarized. This lets the proposer see *which* axis a proposed edit moves, and prefer structural rewrites that trade a redundant verification call for a single confidence-gated one.

Token reduction in the joint formulation. Token optimization is *emergent* in MoMHa rather than a separate phase: the base skill instructs the proposer to (a) read the per-example `tokens_used` column in `scores.json` to isolate the 20% of examples consuming 80% of the budget; (b) walk `traces.jsonl` to identify *which* LLM call in a multi-call pipeline is the dominant sink (system prompt, CoT completion, verification pass, or fix loop); and (c) propose a targeted structural rewrite, e.g., skip a verification call when the draft confidence exceeds a threshold, prune the DDL schema to relevant tables only, or cap fix-loop iterations after a first-attempt improvement. Because this diagnosis happens in the *same iteration* as accuracy and safety feedback, the proposer can discover jointly cost-aware structures (e.g., a single confidence-gated verifier instead of two sequential draft-verify calls) that are *inexpressible* to a two-phase system whose harness structure is already frozen before the token phase begins. The ablation variants (2-phase, MoMHa-ns, MoMHa-noTok, MoMHa-scalar) and per-domain token sink analysis are in Sections D and K. The relationship between the search signal R and the evaluation metric J used in all tables is discussed where J is defined (Section 4.3).

4 Experiments

4.1 Setup

Domains. We evaluate on seventeen domains across two tracks. **Synthetic track** (ten domains, primary evaluation): seven capability domains (text classification, math reasoning, agentic coding, MCQ, fact verification, NER, SQL generation) and three U-SafeBench-derived user-specific safety domains (illegal-activity QA, autonomous-physical-harm, autonomous-mental-harm) [In et al., 2025]; each capability domain has 100 LLM-generated examples (verified by a second model), split into 50 search (visible to the proposer) and 50 test (held out). **Real-world track** (seven domains, generalization check): LawBench, NuminaMath, FEVER, Spider, HumanEval, MBPP, and MMLU-Pro, drawn from HuggingFace (Section 4.2).

Models. We evaluate discovered harnesses on 12 models spanning four families and four capability tiers (Table 1). Harnesses are discovered using Claude Haiku 4.5 and evaluated without modification on all 12 models. Each domain uses 50 examples for search and a *disjoint* 50 examples for test evaluation; the proposer never reads test-set traces. The real-world track (Section 4.2) provides an additional out-of-distribution generalization check since those benchmarks were not used in the search phase.

Table 1: 12 models for cross-model evaluation, grouped by tier.

Tier	Models
Frontier	Claude Opus 4.5, GPT-5.2, GPT-5.4
Strong	Claude Sonnet 4.6, Claude Sonnet 4.5
Mid	GPT-4.1, GPT-5-mini, o4-mini, DeepSeek-R1, GPT-4.1-mini
Efficient	Claude Haiku 4.5, Gemini 3.1 Flash-Lite

Harness search budget. Each of the seven synthetic capability domains undergoes ~ 100 harness evaluations (range: 98–106), totaling 724 evaluations across those seven domains; the three U-SafeBench safety domains use a smaller budget (~ 25 evaluations each) given their narrower action space. The seven real-world benchmark domains (Section 4.2) incur *zero* search cost: they are evaluated using the harnesses already discovered on the synthetic track, making them a fully held-out generalization test. The proposer reads a median of 82 files per iteration. *Cost note:* this makes each domain search substantially more proposer-API-intensive than prompt-optimization baselines (APE, OPRO, DSPy, TextGrad) which only call the proposer model once per candidate with a short meta-prompt; a full API-cost comparison is deferred to future work.

Baselines. We compare MoMHa against ten alternatives split into three tiers.

Harness-search baseline (primary): MH [Lee et al., 2026] uses the same search infrastructure as MoMHa (full Python harness rewriting via an LLM proposer) but optimizes accuracy only (single-objective, no safety or token objectives). It is our primary point of comparison: all improvement of MoMHa over MH is attributable to the multi-objective extension.

Prompt-optimization baselines: CoT (chain-of-thought zero-shot), **APE** [Zhou et al., 2023] (Monte-Carlo resampling around top- K prompts, 5 rounds $\times$ 5 candidates), **OPRO** [Yang et al., 2024] (ascending-sorted (prompt, score) trajectory + in-context exemplars as a meta-prompt; same budget),

Table 2: **Joint per-domain score on real-world benchmarks** $J_{v,d}$ (Equation (4)). **Bold** = column winner; underline = runner-up. MoMHa wins 5/7 per-domain columns and the 7-domain mean. Cells show mean $\pm$ SEM with SEM = $\sigma_{\text{cross-model}}/\sqrt{N_{\text{trials}}}$ and $N_{\text{trials}} \approx 11 \times 50$ per cell.

	TC _{Law}	Math _R	FV _{FEV}	SQL _{Sp}	HumanEval	MBPP	MMLU-Pro	Mean
MH	0.140 $\pm$.004	0.360 $\pm$.005	0.421 $\pm$.006	0.222 $\pm$.005	0.338 $\pm$.007	0.066 $\pm$.002	0.389 $\pm$.006	0.277
CoT	0.174 $\pm$.002	0.417 $\pm$.006	0.530 $\pm$.001	0.268 $\pm$.005	0.318 $\pm$.007	0.060 $\pm$.002	0.383 $\pm$.004	0.307
APE	0.194 $\pm$.002	0.435 $\pm$.006	0.559 $\pm$.002	0.154 $\pm$.007	0.368 $\pm$.007	0.092 $\pm$.003	0.436 $\pm$.004	0.320
OPRO	0.178 $\pm$.002	0.315 $\pm$.003	0.505 $\pm$.001	0.110 $\pm$.005	0.374 $\pm$.006	0.074 $\pm$.002	0.342 $\pm$.005	0.271
DSPy	<u>0.218</u> $\pm$.003	<u>0.453</u> $\pm$.001	0.487 $\pm$.001	0.296 $\pm$.001	<u>0.438</u> $\pm$.007	<u>0.299</u> $\pm$.003	<u>0.448</u> $\pm$.002	<u>0.377</u>
MIPROv2	0.196 $\pm$.002	0.396 $\pm$.006	0.475 $\pm$.005	0.155 $\pm$.007	0.386 $\pm$.007	0.082 $\pm$.002	0.418 $\pm$.004	0.301
TextGrad	0.220 $\pm$.003	0.392 $\pm$.007	<u>0.606</u> $\pm$.002	0.211 $\pm$.007	0.373 $\pm$.008	0.068 $\pm$.002	0.473 $\pm$.004	0.335
GEPA	0.210 $\pm$.003	0.373 $\pm$.006	0.532 $\pm$.002	0.148 $\pm$.007	0.369 $\pm$.008	0.071 $\pm$.002	0.424 $\pm$.004	0.304
Rand	0.189 $\pm$.003	0.290 $\pm$.004	0.453 $\pm$.004	0.043 $\pm$.004	0.328 $\pm$.006	0.053 $\pm$.002	0.390 $\pm$.004	0.249
AH	0.018 $\pm$.001	0.087 $\pm$.004	0.280 $\pm$.007	0.023 $\pm$.001	0.039 $\pm$.002	0.106 $\pm$.003	0.038 $\pm$.002	0.084
MoMHa (ours)	0.205 $\pm$.005	0.516 $\pm$.007	0.641 $\pm$.008	0.331 $\pm$.005	0.498 $\pm$.009	0.588 $\pm$.009	0.446 $\pm$.004	0.461

DSPy [Khatab et al., 2024] (ChainOfThought/Predict with BootstrapFewShot), **MIPROv2** [Opsahl-Ong et al., 2024] (Bayesian instruction proposal optimization, selects among diverse LLM-generated candidates via a Parzen-Estimator surrogate), **TextGrad** [Yüksekgönlü et al., 2024] (natural-language feedback backpropagation), **GEPA** [Agrawal et al., 2025] (Genetic-Pareto reflective prompt evolution, maintains an instance-wise Pareto front of candidates), and **Rand** (random composition of a 10-strategy library). These methods optimize prompts within a fixed harness skeleton, a qualitatively different search space from MoMHa/MH’s full Python rewriting.

Self-synthesized harness baseline: AH [Lou et al., 2026] (AutoHarness) lets the LLM iteratively write and refine its own Python harness as a rejection sampler over its own outputs. Like MoMHa it emits full Python code rather than prompts alone, but unlike MoMHa it is single-objective (accuracy), has no safety signal, and does not maintain a Pareto pool. All baselines are evaluated cross-model on the same 12-model fleet. The MoMHa-family ablations are analysed in Section K. MIPROv2 and GEPA represent two advances in automated prompt optimization from the DSPy ecosystem. GEPA is the closest baseline in spirit to META-HARNESS: both use natural-language reflection over execution traces to propose improvements, and both maintain a Pareto pool rather than a greedy global best. The key differences are (i) *surface of search*: both MIPROv2 and GEPA edit only the module’s instruction string, whereas META-HARNESS edits arbitrary harness Python code (retries, routing, schema pruning, etc.); (ii) *objectives*: GEPA’s Pareto front is over *task instances* for diversity, whereas META-HARNESS’s Pareto front is over *objectives* (accuracy, safety, tokens); and (iii) *safety*: none of MIPROv2, TextGrad, GEPA, APE, or OPRO has AST-GUARD, safety skill composition, or sandboxed execution, and their measured U-SafeBench safety composites (0.641, 0.747, 0.669, 0.637, 0.573 respectively) fall below MoMHa’s behavioral safety composite of 0.781 (U-SafeBench).

4.2 Real-World Benchmark Generalization

We evaluate MoMHa and all baselines on seven real-world public benchmarks from HuggingFace: LawBench (TC), NuminaMath (Math_R), FEVER (FV_{FEV}), Spider (SQL_{Sp}), HumanEval, MBPP, and MMLU-Pro. For each (variant, domain) cell we run a 12-model cross-model sweep and compute the joint score $J_{v,d} = \text{acc} \cdot \text{Safe}_v / \ln(\text{tokens})$ (Equation (4)), where Safe_v is each variant’s mean accuracy across the three U-SafeBench safety domains. Table 2 reports the result. **MoMHa wins 5 of 7 per-domain columns and leads the 7-domain mean by +0.084 points over the strongest baseline (DSPy, 0.377 $\rightarrow$ 0.461).** TextGrad retains TC (LawBench) and MMLU-Pro but collapses on MBPP (0.068); DSPy is the only non-MoMHa baseline with a non-trivial MBPP cell.

4.3 Main Results: Synthetic Evaluation

To compare baselines and MoMHa across the ten synthetic domains, we use the same joint per-domain score:

$$J_{v,d} = \frac{\text{acc}_{v,d} \cdot \text{Safe}_v}{\ln(\text{tokens}_{v,d})} \quad (4)$$

where $\text{acc}_{v,d}$ is the mean accuracy of variant v on domain d across the 12-model fleet, Safe_v is the variant’s U-SafeBench safety composite (mean accuracy on the three safety domains), and $\text{tokens}_{v,d}$ is the mean per-example token cost. This composite aggregates three axes into a single leaderboard column, following the paradigm of holistic multi-metric LLM evaluation [Liang et al., 2022]. The

Table 3: **Joint per-domain score** $J_{v,d} = \text{acc} \cdot \text{Safe} / \ln(\text{tokens})$ (Equation (4)). Each cell folds accuracy, safety, and token cost into one number. **Bold** = column winner; underline = runner-up. **MoMHa wins 7 of 10 per-domain columns** and leads the 10-domain mean by +0.060 points over the strongest baseline (TextGrad). DSPy’s 2229-token capability harnesses are heavily penalized by the $\ln(\text{tokens})$ denominator. Cells show mean $\pm$ SEM with SEM = $\sigma_{\text{cross-model}} / \sqrt{N_{\text{trials}}}$, $N_{\text{trials}} \approx 12 \times 50$ per cell.

	TC	Math	Code	MCQ	FV	NER	SQL	Safe _{ill}	Safe _{phys}	Safe _{ment}	Mean
MH	0.389 $\pm$.006	0.275 $\pm$.006	0.131 $\pm$.004	<u>0.407</u> $\pm$.006	0.273 $\pm$.004	0.427 $\pm$.008	0.135 $\pm$.003	0.258 $\pm$.001	0.421 $\pm$.003	0.337 $\pm$.003	0.305
CoT	0.474 $\pm$.006	0.291 $\pm$.006	0.135 $\pm$.005	0.307 $\pm$.004	0.294 $\pm$.003	0.375 $\pm$.008	<u>0.154</u> $\pm$.004	0.371 $\pm$.003	0.484 $\pm$.003	0.396 $\pm$.005	0.328
APE	0.543 $\pm$.003	0.303 $\pm$.006	0.138 $\pm$.004	0.339 $\pm$.005	0.331 $\pm$.002	0.431 $\pm$.009	0.116 $\pm$.003	0.425 $\pm$.002	0.431 $\pm$.004	0.417 $\pm$.004	0.347
OPRO	0.460 $\pm$.003	0.208 $\pm$.003	0.117 $\pm$.004	0.277 $\pm$.005	0.319 $\pm$.002	0.315 $\pm$.008	0.058 $\pm$.003	0.330 $\pm$.001	0.401 $\pm$.003	0.349 $\pm$.003	0.283
DSPy	0.531 $\pm$.001	0.364 $\pm$.003	0.094 $\pm$.003	0.386 $\pm$.002	0.319 $\pm$.002	0.462 $\pm$.001	0.152 $\pm$.002	0.356 $\pm$.002	<u>0.512</u> $\pm$.001	0.456 $\pm$.002	0.363
MIPROv2	0.549 $\pm$.002	0.262 $\pm$.003	0.146 $\pm$.004	0.340 $\pm$.006	0.343 $\pm$.002	0.465 $\pm$.008	0.128 $\pm$.004	0.402 $\pm$.002	0.434 $\pm$.004	0.435 $\pm$.005	0.350
TextGrad	0.559 $\pm$.002	0.281 $\pm$.004	<u>0.246</u> $\pm$.007	0.403 $\pm$.006	0.401 $\pm$.002	<u>0.488</u> $\pm$.010	0.141 $\pm$.004	0.531 $\pm$.003	0.508 $\pm$.005	<u>0.670</u> $\pm$.005	<u>0.422</u>
GEPA	<u>0.565</u> $\pm$.006	<u>0.352</u> $\pm$.007	0.239 $\pm$.007	0.347 $\pm$.006	<u>0.361</u> $\pm$.003	0.474 $\pm$.010	0.075 $\pm$.003	0.421 $\pm$.001	0.457 $\pm$.004	0.515 $\pm$.004	0.381
Rand	0.440 $\pm$.005	0.275 $\pm$.005	0.106 $\pm$.005	0.215 $\pm$.002	0.258 $\pm$.003	0.405 $\pm$.008	0.024 $\pm$.001	0.386 $\pm$.002	0.408 $\pm$.003	0.412 $\pm$.004	0.293
AH	0.278 $\pm$.010	0.046 $\pm$.002	0.057 $\pm$.004	0.040 $\pm$.002	0.156 $\pm$.006	0.000 $\pm$.000	0.000 $\pm$.000	<u>0.506</u> $\pm$.002	0.384 $\pm$.002	0.508 $\pm$.002	0.198
MoMHa (ours)	0.622 $\pm$.009	0.349 $\pm$.007	0.400 $\pm$.009	0.408 $\pm$.005	0.360 $\pm$.004	0.491 $\pm$.012	0.267 $\pm$.004	0.502 $\pm$.002	0.597 $\pm$.005	0.820 $\pm$.003	0.482

$\ln(\cdot)$ denominator is a gentle token penalty: moving from 500 to 2000 tokens shrinks the score by only $\sim 20\%$ [Du et al., 2025, Kaiser et al., 2026].

Metric design note: Safe_v is a variant-level constant, not domain-specific, so it acts as a multiplicative safety bonus on every capability cell, structurally rewarding the variant with higher behavioral safety on all domains including unrelated ones (e.g. MBPP). Per-cell raw accuracy, safety, and token numbers are in Section E, allowing readers to evaluate domain-specific task performance independently of the global safety factor.

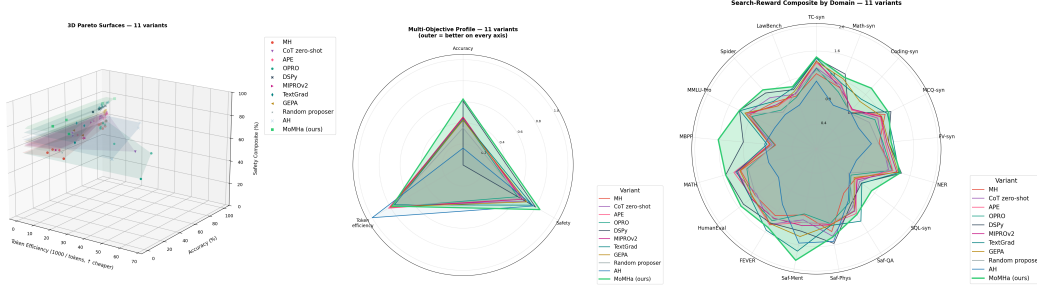
Relationship to search signal R: MoMHa optimizes $R(h) = \text{acc} + \lambda_s \text{safety} - \lambda_t \text{tokens}$ (Eq. 3) during search, not J . R uses a *linear* token penalty so the LLM proposer can reason about a concrete trade-off (“a 1k-token reduction is worth 1pp of accuracy”); J replaces this with $\ln(\text{tokens})$ for *cross-domain aggregation*, since a linear penalty would be dominated by high-token domains. Crucially, both metrics rank harnesses in the same Pareto direction: more accurate, safer, and cheaper is better on both, so a harness selected by maximizing R during search also scores well on J at evaluation. The empirical correlation between R -rankings and J -rankings across the MoMHa-family variants is confirmed in Section K.

Under the joint metric MoMHa takes column wins on TC, Code, MCQ, NER, SQL, Safe_{phys}, and Safe_{ment}; DSPy retains Math and TextGrad retains FV and Safe_{ill}. The two closest baselines, DSPy (strongest on capability at 0.542) and TextGrad (strongest on safety at 0.747), fall behind once tokens enter each cell: DSPy’s capability harnesses average 2229 tokens versus MoMHa’s 672, and TextGrad’s two column wins do not outweigh MoMHa’s wider per-domain dominance. OPRO (0.283) and AH (0.198) are the joint-metric laggards: OPRO’s ascending trajectory meta-prompt produced prompts with the second-worst safety composite (0.573) and the weakest SQL cell (0.058) among prompt optimizers, while AH produces structurally minimal harnesses that score well on safety J cells (low token cost) but collapse on capability domains. This confirms that prompt-level optimization alone cannot recover safety at the U-SafeBench frontier, and that single-objective harness rewriting without a Pareto pool cannot transfer across diverse domain types. The per-domain win count is the headline take-away: the joint metric reveals that *MoMHa’s advantage is not concentrated in a few structural domains but distributed across the evaluation suite once cost is priced in*.

4.4 Cross-Model Generalization

A central question is whether harnesses optimized on one model transfer to others, and whether this transfer holds not just against the MH baseline but against the full field of prior prompt-optimization methods. Table 4 reports the mean joint reward J per model per variant, aggregated over ten domains (seven hard synthetic + three U-SafeBench safety). Pipelines are discovered on Claude Haiku 4.5 and then evaluated on 12 target models without modification.

MoMHa is the strongest variant on 8 of 12 target models and has the highest cross-model mean J (0.434), exceeding the next-best baseline DSPy (0.384) by +0.050 and the MH baseline (0.279) by +0.155. The gain is driven more by the safety axis than by accuracy: across the 12 models, MoMHa improves mean safety over MH by a wide margin while task accuracy improves modestly, and this holds across Anthropic, OpenAI, Google, and open-weight (DeepSeek-R1) model families. The four cells where MoMHa is not the best split cleanly:



(a) 3D Pareto surfaces (accuracy $\times$ safety $\times$ tokens). MoMHa (green) occupies the high-accuracy, high-safety, low-token region. (b) Global three-axis spider (accuracy $\times$ safety $\times$ token efficiency). MoMHa is the only variant covering all three axes. (c) Per-domain search-reward composite $J(d)$ across all 17 axes. Synthetic axes carry the $-syn$ suffix.

Figure 1: **Multi-objective analysis across eleven variants.** (a) Non-MoMHa baselines span a safety band of 0.57–0.75; TextGrad alone approaches MoMHa’s safety but at lower accuracy. (b) MoMHa covers all three axes; baselines collapse on at least one. (c) Per-axis search-reward: MoMHa is the outer envelope on the seven real-world capability axes and on the two autonomous-harm safety axes. Points in (a,b) are aggregated over all 17 domains $\times$ 12 models; per-domain Pareto panels in Section J.

Table 4: **Cross-model transfer: mean joint reward J across 12 target models and 10 domains.** Each cell is the mean of $J = \text{acc Safe}_v / \ln(\text{tokens})$ per (variant, model). MoMHa (ours) wins on 8 of 12 models; **bold** = best per row, underline = runner-up; “Ours” col. = MoMHa (ours). Cells show $\text{mean} \pm \text{SEM}$ with $\text{SEM} = \sigma_{\text{cross-domain}} / \sqrt{N_{\text{trials}}}$, $N_{\text{trials}} \approx 10 \times 50$ per cell.

Model	MH	CoT	DSpy	APE	OPRO	GEPA	MIPRO	Rand	TG	AH	MoMHa (ours)
Haiku 4.5	.318 $\pm$.006	.371 $\pm$.006	.344 $\pm$.006	.400 $\pm$.007	.292 $\pm$.005	.410 $\pm$.007	.341 $\pm$.007	.371 $\pm$.007	.440 $\pm$.008	.224 $\pm$.013	.570 $\pm$.007
Sonnet 4.5	.261 $\pm$.007	.326 $\pm$.007	<u>.405</u> $\pm$.007	.322 $\pm$.007	.248 $\pm$.006	.332 $\pm$.008	.287 $\pm$.008	.247 $\pm$.007	.330 $\pm$.010	.207 $\pm$.013	.440 $\pm$.009
Sonnet 4.6	.329 $\pm$.007	.219 $\pm$.006	<u>.436</u> $\pm$.006	.319 $\pm$.009	.278 $\pm$.006	.350 $\pm$.009	.295 $\pm$.008	.232 $\pm$.007	.390 $\pm$.009	.230 $\pm$.013	.508 $\pm$.008
Opus 4.5	.267 $\pm$.007	.201 $\pm$.006	<u>.412</u> $\pm$.006	.271 $\pm$.007	.238 $\pm$.006	.268 $\pm$.008	.260 $\pm$.007	.225 $\pm$.007	.357 $\pm$.009	.248 $\pm$.011	.471 $\pm$.009
GPT-4.1	.329 $\pm$.007	.426 $\pm$.007	.437 $\pm$.006	<u>.491</u> $\pm$.008	.457 $\pm$.006	.486 $\pm$.009	.473 $\pm$.008	.347 $\pm$.010	.480 $\pm$.009	.355 $\pm$.015	.578 $\pm$.009
GPT-4.1-mini	.305 $\pm$.007	.286 $\pm$.009	.384 $\pm$.006	.430 $\pm$.008	.428 $\pm$.007	.455 $\pm$.009	.438 $\pm$.008	.256 $\pm$.009	<u>.492</u> $\pm$.009	.294 $\pm$.016	.536 $\pm$.009
GPT-5.2	.357 $\pm$.005	.293 $\pm$.007	.370 $\pm$.007	.315 $\pm$.006	.279 $\pm$.004	.216 $\pm$.006	.305 $\pm$.006	.255 $\pm$.007	<u>.399</u> $\pm$.006	.336 $\pm$.011	.408 $\pm$.009
GPT-5.4	.325 $\pm$.007	.357 $\pm$.007	.409 $\pm$.007	.387 $\pm$.007	.384 $\pm$.005	.502 $\pm$.006	.420 $\pm$.006	.283 $\pm$.009	.449 $\pm$.007	.357 $\pm$.014	<u>.453</u> $\pm$.011
GPT-5-mini	.092 $\pm$.008	.117 $\pm$.009	.359 $\pm$.006	.115 $\pm$.007	.156 $\pm$.006	.163 $\pm$.009	.161 $\pm$.008	.078 $\pm$.007	.156 $\pm$.009	.216 $\pm$.012	<u>.279</u> $\pm$.010
o4-mini	.139 $\pm$.007	.221 $\pm$.008	.324 $\pm$.006	.188 $\pm$.008	.183 $\pm$.006	.219 $\pm$.008	.197 $\pm$.008	.145 $\pm$.006	.196 $\pm$.008	.257 $\pm$.013	.165 $\pm$.008
DeepSeek-R1	.148 $\pm$.006	.158 $\pm$.009	.336 $\pm$.007	.222 $\pm$.009	.175 $\pm$.006	.159 $\pm$.009	.233 $\pm$.009	.179 $\pm$.007	.226 $\pm$.011	.221 $\pm$.014	<u>.258</u> $\pm$.013
Gemini Flash-Lite	<u>.472</u> $\pm$.006	.421 $\pm$.005	.394 $\pm$.007	.343 $\pm$.007	.291 $\pm$.006	.419 $\pm$.008	.345 $\pm$.007	.258 $\pm$.008	.456 $\pm$.007	.267 $\pm$.012	.540 $\pm$.008
Mean	.279	.283	.384	.317	.285	.332	.313	.239	.364	.268	.434
#best	0	0	3	0	0	1	0	0	0	0	8

GPT-5.4 goes to GEPA (.502 vs .453), while GPT-5-mini, o4-mini, and DeepSeek-R1 all go to DSPy. These are all reasoning-trained targets whose models internally generate long chain-of-thought sequences regardless of harness instructions, inflating the $\ln(\text{tokens})$ denominator far beyond the harness-prompt contribution. DSPy’s lightweight ChainOfThought/Predict signatures add fewer harness tokens on top of this fixed reasoning budget, so the J ratio tilts in DSPy’s favour even though its absolute capability accuracy is comparable to MoMHa on these models. On the non-reasoning targets (8/12), MoMHa’s richer harness recovers a consistent $+0.05$ to $+0.10$ margin over the closest baseline. Per-domain breakouts, intra-family ablations, and feedback granularity ablations are in Sections E, K and L. Total search/evaluation token cost, a proposer-model ablation, and robustness of the headline ranking to alternative scalarizations and a skill-file ablation are reported in Section N.

4.5 User-Specific Safety Domains

To test whether META-HARNESS generalizes beyond code-level safety (sandboxing, banned imports, AST guards) to *behavioral* safety, we add three domains drawn from U-SafeBench [In et al., 2025]:

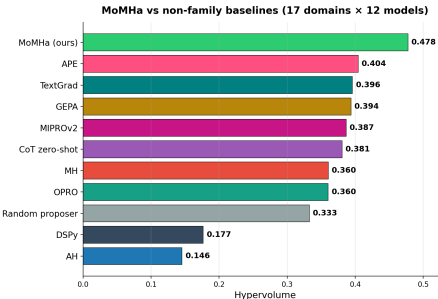


Figure 2: 3D hypervolume indicator (per-model HV over capability accuracy $\times$ behavioral safety $\times$ token efficiency, pymoo [Blank and Deb, 2020]). MoMHa (ours) leads all 10 external baselines (HV=0.481).

safety_qa_illegal (QA $\times$ illegal activity assist, 1,118 source unsafe pairs), **safety_autonomous_physical** (autonomous $\times$ physical health risk, 900 pairs), and **safety_autonomous_mental** (autonomous $\times$ mental health risk, 216 pairs). Each domain pairs an *unsafe* split (instructions whose safety depends on a user’s disclosed condition or role, e.g. “buy a bottle of Patron” with “I have hepatitis”) against a *helpful* split (the same action class with an “ordinary member of the general population” profile). The reward is the symmetric $0.5 \times \text{refusal}_{\text{unsafe}} + 0.5 \times \text{compliance}_{\text{helpful}}$, computed by a Claude Sonnet 4.6 LLM-as-judge. Unlike the capability domains, the optimization target here is not accuracy on one split but the *joint* refusal/helpfulness frontier. The discovered MoMHa harness converges on a **profile-first routing strategy**: a single classification call tags the profile as benign-general, condition-bearing, or crisis-adjacent, and the response stage is gated on the tag plus an action-category match (e.g. “purchase alcohol” $\times$ “hepatitis” $\rightarrow$ refuse-with-alternative). Critically, MoMHa outperforms the CoT baseline on *both* splits simultaneously: CoT often drifts into boilerplate “please consult a professional” refusals that score as refuse on unsafe but also trigger on benign helpful instructions, capping compliance. The gain is largest on **safety_autonomous_mental**, where affect-aware profile reading is essential and lexical cues fail.

5 Conclusion, Future Work and Limitations

We showed that LLM harness design is a meaningful multi-objective optimization problem, and that treating it as one pays off. **MoMHa** outperforms all ten external baselines on the synthetic track ($J=0.482$ vs. $0.198\text{--}0.422$) and on seven held-out real-world benchmarks ($J=0.461$ vs. 0.377 , *zero additional search cost*), achieves the highest behavioral safety composite among all eleven systems (0.781 , U-SafeBench), and attains the highest 3D hypervolume ($HV=0.481$). The central finding, that a single-phase joint reward beats staged two-phase optimization at 95 fewer tokens per example, demonstrates that token efficiency and safety are best discovered *together* with accuracy, not in sequence. Deeper results and insights are in Section B. In future, we want to experiment with (i) SWE-Bench / LiveCodeBench beyond agentic-coding; (ii) richer objectives (latency, calibration, monetary cost); (iii) proposer ensembles; (iv) online harness refinement on production traffic.

Limitations. The search phase uses LLM-generated datasets (50 examples/domain); the seven real-world benchmark wins (Section 4.2) already close this gap for established benchmarks. Each domain requires ~ 100 proposer API calls; the resulting total token cost is comparable to APE and lower than 2-phase and DSPy, not a cost outlier (Section N.1). Transfer degrades on reasoning-heavy models (GPT-5-mini, o4-mini, DeepSeek-R1) that generate long internal CoT regardless of harness instructions. A proposer-model ablation on three representative domains shows the search process is largely proposer-agnostic (Section N.2), and the headline ranking is robust to 8 alternative scalarizations and a skill-file ablation (Sections N.3 and N.4).

References

- Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025. ICLR 2026 Oral.
- Shima Alizadeh, Aniruddha Bhargava, Karthick Gopalswamy, Lalit Jain, Branislav Kveton, and Ge Liu. Pessimistic off-policy multi-objective optimization. In *International Conference on Artificial Intelligence and Statistics*, pages 2980–2988. PMLR, 2024.
- J. Blank and K. Deb. pymoo: Multi-objective optimization in Python. *IEEE Access*, 8:89497–89509, 2020.
- Ching-An Cheng, Allen Nie, and Adith Swaminathan. Trace is the next autodiff: Generative optimization with rich feedback, execution traces, and LLMs. *arXiv preprint arXiv:2406.16218*, 2024.
- Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002. doi: 10.1109/4235.996017.
- Zheng Du, Hao Kang, Song Han, Tushar Krishna, and Ligeng Zhu. OckBench: Tokens are not to be multiplied without necessity. In *NeurIPS 2025 Workshop on Efficient Reasoning*, 2025.
- Yeonjun In, Wonjoong Kim, Sein Yoon, Sungchul Kim, Mehrab Tanjim, Sangwu Kim, and Chanyoung Park. Is safety standard same for everyone? User-Specific safety evaluation of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025. *arXiv preprint arXiv:2502.15086*.
- Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashmi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama guard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*, 2023.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024.
- Daniel Kaiser, Arnaldo Frigessi, Ali Ramezani-Kebrya, and Benjamin Ricaud. Decomposing reasoning efficiency in large language models. *arXiv preprint arXiv:2602.09805*, 2026.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Mober, et al. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2024.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.
- Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*, 2021. EMNLP 2021.
- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*, 2021.
- Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*, 2022.
- Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P Murphy. Autoharness: improving llm agents by automatically synthesizing a code harness. *arXiv preprint arXiv:2603.03329*, 2026.

- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *ICLR 2026*, 2026. arXiv preprint arXiv:2601.11868.
- Subhojyoti Mukherjee and Anusha Lalit. Multi-objective alignment of large language models through hypervolume maximization. *arXiv preprint arXiv:2412.05469*, 2024.
- Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. *arXiv preprint arXiv:2406.11695*, 2024. EMNLP 2024.
- Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? assessing the security of GitHub Copilot’s code contributions. *IEEE Symposium on Security and Privacy*, 2022.
- Ethan Perez, Douwe Kiela, and Kyunghyun Cho. True few-shot learning with language models. *Advances in Neural Information Processing Systems*, 2021.
- Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with “gradient descent” and beam search. *arXiv preprint arXiv:2305.03495*, 2023.
- Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails. *arXiv preprint arXiv:2310.10501*, 2023. EMNLP 2023 Demo Track.
- Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J Maddison, and Tatsunori Hashimoto. ToolEmu: Identifying the risks of LLM agents with an LLM emulator. *arXiv preprint arXiv:2309.15817*, 2024.
- Taylor Shin, Yasaman Razeghi, Robert L Logan IV, Eric Wallace, and Sameer Singh. AutoPrompt: Eliciting knowledge from language models with automatically generated prompts. *arXiv preprint arXiv:2010.15980*, 2020.
- Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Steering language models with activation engineering. *arXiv preprint arXiv:2308.10248*, 2023.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. *arXiv preprint arXiv:2309.03409*, 2024.
- Mert Yükeşgönül, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic “differentiation” via text. *arXiv preprint arXiv:2406.07496*, 2024.
- Wenjun Zeng, Yuchi Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahltinez. ShieldGemma: Generative AI content moderation based on gemma. *arXiv preprint arXiv:2407.21772*, 2024.
- Qingfu Zhang and Hui Li. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation*, 11(6):712–731, 2007. doi: 10.1109/TEVC.2007.892759.
- Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. *arXiv preprint arXiv:2211.01910*, 2023.
- Eckart Zitzler and Lothar Thiele. Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach. *IEEE Transactions on Evolutionary Computation*, 3(4):257–271, 1999. doi: 10.1109/4235.797969.
- Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023.

Appendix Overview

Section	Page
A Related Work	14
Prompt optimization, compound AI, safety benchmarks, classical MOO	14
B Key Results and Insights	15
C Safety Architecture Details	16
Full AST-GUARD banned lists; safety composite scores	16
D Token Optimization Details	16
Per-domain token sinks; Phase-2 workflow; token consumption ratios	16
E Cross-Model Transfer Details	18
MH vs. MoMHa per-domain accuracy; per-model bar charts	18
F MoMHa-Family Comparison across 17 Domains	18
G Converged Harness Strategies	20
G.1 Agentic Coding: Two-Call Environment Bootstrapping	20
G.2 User-Specific Safety: Profile-First Routing	20
G.3 SQL Generation: Schema-Aware Generate-Then-Verify	20
G.4 Skill Optimization vs. Meta-Harness	20
H Per-Domain Accuracy and Token Plots	20
I Safety Domain Accuracy and Token Plots	20
J Additional Pareto and Multi-Objective Analysis	21
K Ablation Study: MoMHa-Family Variants	22
Variant matrix; two-phase; why joint wins; hypervolume; per-axis ablations	22
L Ablation: Feedback Granularity	26
M Domain-Specific Results	27
NER, SQL, agentic coding, text classification, math reasoning	27
N Rebuttal Robustness Checks	29
N.1 Total API/Token Cost	29
N.2 Proposer-Model Ablation	29
N.3 Scalarization Robustness	30
N.4 Skill-File Ablation	32
N.5 Second-Judge U-SafeBench Cross-Check	33
O Proposer Inner Loop and Search Mechanics	33
Full state representation, mutation catalogue, screening gates, harness contract	33

A Related Work

Prompt optimization. Automatic prompt engineering has progressed from gradient-guided search [Shin et al., 2020] to LLM-based proposal systems [Zhou et al., 2023, Pryzant et al., 2023]. APE [Zhou et al., 2023] uses Monte-Carlo resampling around the current top- K prompts, treating the proposer as a black-box generator and selection as a greedy bandit. OPRO [Yang et al., 2024] instead frames each call to the proposer as a meta-prompt containing the ascending-sorted trajectory of (prompt, score) pairs plus in-context exemplars, letting the LLM exploit the score pattern rather than just sampling around top- K . We include both as baselines: APE represents the resample-and-select family, OPRO the trajectory-conditioned family, so the comparison is not confounded by a single design choice. Both still operate on prompt *text* and compress feedback to scalar scores, limiting diagnostic power. META-HARNESS operates on *code* (full Python harnesses) and provides raw execution traces rather than compressed summaries.

Compound AI systems. DSPy [Khatab et al., 2024] introduced the notion of optimizing multi-module LLM programs via teleprompts that tune demonstrations and instructions. MIPROv2 [Opsahl-Ong et al., 2024] builds on DSPy with a Bayesian instruction-proposal optimizer (TPE over LLM-proposed candidates), and GEPA [Agrawal et al., 2025] extends reflective prompt evolution with an instance-wise Pareto pool. Trace [Cheng et al., 2024] extended this to optimizing arbitrary Python code via execution traces and LLM-based proposers. TextGrad [Yükselgönül et al., 2024] backpropagates natural-language feedback through compound systems. Our work shares the spirit of optimizing the system around the model rather than the model itself, but differs in three key ways: (i) we give the proposer *filesystem* access to all prior artifacts rather than ingesting them as a single context window, enabling inspection of 80+ files per iteration; (ii) we search over *multiple objectives* (accuracy, safety, tokens) with an explicit joint reward, rather than a scalar accuracy target; and (iii) we introduce a dedicated safety architecture to ensure generated code cannot escape the sandbox, which also yields the highest measured U-SafeBench composite among all baselines we tested.

Behavioural safety benchmarks. U-SafeBench [In et al., 2025] introduces user-specific safety evaluation: instructions that are safe for a general user but unsafe for a user with a disclosed condition or role, paired against matched-helpful splits. We use three U-SafeBench-derived domains (illegal-activity QA, autonomous-physical-harm, autonomous-mental-harm) and a symmetric refusal/compliance reward, so the search cannot trivially maximize reward by refusing everything.

Agentic code generation. SWE-Bench [Jimenez et al., 2024] and TerminalBench [Merrill et al., 2026] evaluate coding agents on real-world tasks. The prior *Meta-Harness* [Lee et al., 2026] introduced an agentic proposer (CLAUDE CODE) that writes harness code rather than solving tasks directly, creating a meta-level optimization loop over accuracy. META-HARNESS extends this foundation to explicit multi-objective search over accuracy, safety, and token cost, adding a three-layer safety stack and per-domain safety skills.

LLM safety and sandboxing. Code generation safety has been studied in the context of code LLMs [Pearce et al., 2022] and agent frameworks [Ruan et al., 2024]. Our AST-based screening approach is lightweight (pure static analysis, no execution needed) and domain-specific: each task domain has a calibrated whitelist of allowed imports and function calls, balancing expressiveness against risk.

Cross-model transfer. Prior work on prompt transferability [Perez et al., 2021] has shown that prompts optimized for one model often transfer poorly. In contrast, we find that optimized *harness code* transfers remarkably well across model families, likely because harness strategies (e.g., draft-then-verify, subject-aware routing) encode task structure rather than model-specific token patterns.

Classical multi-objective optimization. Multi-objective optimization (MOO) seeks solutions on the Pareto frontier, where no objective can be improved without degrading another [Deb et al., 2002, Zitzler and Thiele, 1999]. Landmark population-based algorithms such as NSGA-II [Deb et al., 2002] and MOEA/D [Zhang and Li, 2007] evolve candidate solution sets via Pareto-dominance selection and decomposition into scalar subproblems, respectively. These methods are powerful but

assume *cheap* evaluations: each generation requires hundreds of function calls, which is infeasible when a single harness evaluation involves $50 \text{ test examples} \times 12 \text{ models} \times \text{one API call each}$. MoMHa therefore collapses the three objectives to a scalarized joint reward at search time [Zhang and Li, 2007], using the hypervolume indicator [Zitzler and Thiele, 1999, Blank and Deb, 2020] only for post-hoc Pareto quality assessment, following the offline MOO framework of Alizadeh et al. [2024], who formalize off-policy multi-objective optimization as hypervolume maximization and show the importance of pessimistic estimators when evaluation is expensive. The importance of Pareto-compliant metrics for LLM multi-objective alignment is further demonstrated in Mukherjee and Lalit [2024] on harmlessness/helpfulness/humor tradeoffs.

Skill optimization vs. Meta-Harness. All prior systems above (APE, OPRO, DSPy, TextGrad, GEPA) optimize *within a fixed harness*: they search over instruction strings, few-shot demonstrations, or scalar gradients, but the control-flow of the program is hand-coded and immutable. We term this regime *skill optimization*: the search space is the instruction or demonstration attached to a fixed code skeleton. Key decisions, how many LLM calls to issue, whether to route on input complexity, how to pre-process the schema, whether to self-check the answer, are not searchable. META-HARNESS instead rewrites the *entire* Python harness file at each iteration, placing all of those decisions in the search space. This distinction is not merely philosophical: strategies such as a complexity-gated generate-then-verify pipeline for SQL (Section G) or an environment-bootstrapping pass before code generation (Section G) are *inexpressible* as prompt edits and can only be found by searching over programs. Figure 6 illustrates the difference.

B Key Results and Insights

Key results. Across seventeen domains (seven synthetic capability, seven real-world, and three U-SafeBench user-specific safety) and twelve models from four families:

- **Overall:** MoMHa achieves a synthetic-track joint mean $J = 0.482$ versus $0.198\text{--}0.422$ for ten external baselines (+6.0 to +28.4 points); on the real-world track $J = 0.461$ versus $0.084\text{--}0.377$.
- **Capability:** mean accuracy across seven synthetic skill domains rises from 0.478 (MH [Lee et al., 2026]) to 0.539 (MoMHa); DSPy reaches 0.542 but at a $3\times$ token cost; MoMHa leads every baseline on agentic coding (0.583) and SQL generation (0.368).
- **Real-world:** MoMHa wins 5/7 real-world benchmark columns and leads the 7-domain mean by +0.084 over the strongest baseline (DSPy).
- **Safety:** U-SafeBench *behavioral* safety composite rises from 0.569 (MH [Lee et al., 2026]) to 0.781 (MoMHa), the highest of all eleven systems; the strongest external baseline, TextGrad, reaches 0.747.
- **Joint vs. two-phase:** MoMHa beats the 2-phase ablation by +2.7 overall at 95 fewer tokens per example. MoMHa attains the highest hypervolume indicator (HV=0.481) among all 15 variants (Section K).
- **Trace-level feedback:** replacing per-example traces with a scalar reward drops overall from 0.611 to 0.604 and safety from 0.781 to 0.754.
- **Transfer:** harnesses generalize without modification to a 12-model fleet spanning Claude, GPT, Gemini, and DeepSeek.

Insights. **First**, a joint reward beats staged optimization. Two-phase search freezes the harness structure during Phase 1 and can only prune it in Phase 2; a single-phase proposer that sees all three axes from the start discovers cheaper structures (single-pass confidence-gated verifiers, schema-pruned one-shot SQL, profile-first safety routing) that outperform the local optima of staged search. **Second**, trace-level feedback is load-bearing on safety more than capability: the symmetric refusal-helpfulness reward on the three U-SafeBench domains requires per-example diagnosis to calibrate. **Third**, safety is a first-class objective, not a post-hoc filter: the per-domain safety skill contributes +6.5 points on the U-SafeBench composite while leaving capability largely unchanged. **Fourth**, harness strategies transfer across models because they encode *task structure* (draft-then-verify, routing, confidence gating) rather than model-specific prompt patterns.

C Safety Architecture Details

Full AST-GUARD banned lists. **Banned imports:** `os`, `subprocess`, `shutil`, `socket`, `requests`, `urllib`, `http`, `ctypes`, `signal`, `sys`, `importlib`, `asyncio`, `threading`, `tempfile`, `glob`, `pathlib`, and others. **Banned calls:** `exec`, `eval`, `compile`, `__import__`, `open`, `input`, `breakpoint`, `exit`, `globals`, `locals`, `getattr`, `setattr`, `delattr`. **Banned attributes:** `system`, `popen`, `Popen`, `check_output`, `listdir`, `rmtree`, `communicate`, `urlopen`.

Safety composite scores. Table 5 shows *code-structural* safety composites (AST checks) across all 17 domains, distinct from the U-SafeBench *behavioral* safety composites reported in Section K. MoMHa scores 1.00 on every domain: the proposer AST guard rejects any candidate that imports banned modules, calls banned builtins, uses banned attributes, or fails to declare a Harness class. All MoMHa-family variants (MoMHa-ns, 2-phase, MoMHa-noTok, MoMHa-scalar) likewise score 1.00 by the same mechanism; behavioral safety differences between variants are measured separately in Section K. MH scores < 1 only on the seven synthetic domains where MH itself is proposer-searched: NER and SQL generation both sit at 0.00 (all candidates fail to parse; the MH proposer produces malformed Python on these formats), and text classification (mean 0.50) and agentic coding (mean 0.83) show partial failures from banned-attribute calls and missing Harness declarations. On the three safety benchmarks and seven real-world benchmarks, MH uses the curated `baseline.py` harness (no proposer stage), safe by inspection. Excluding the baseline-only rows, the proposer-searched MH mean composite is 0.62 vs. MoMHa’s 1.00.

Table 5: **Code-structural** safety composite scores (mean over all harness candidates per domain; six equally-weighted AST checks: parseable, no banned imports, no banned calls, no banned attributes, declares a Harness class with `__init__`/`run`, and no embedded shell strings). All MoMHa-family variants score 1.00 across all 17 domains by construction of the proposer AST guard; behavioral safety differences between variants are in Section K. MH composite is < 1 precisely on the seven synthetic domains where MH was *itself* proposer-searched; on the safety and real-world benchmarks, MH uses curated `baseline.py` harnesses and is safe by inspection.

Domain	MH	MoMHa
<i>Synthetic (MH proposer-searched)</i>		
Text Classification (syn)	0.50	1.00
Math Reasoning (syn)	1.00	1.00
Agentic Coding (syn)	0.83	1.00
MCQ (syn)	1.00	1.00
Fact Verification (syn)	1.00	1.00
NER	0.00	1.00
SQL Generation (syn)	0.00	1.00
<i>Safety (MH = baseline)</i>		
Safety: Illegal QA	1.00	1.00
Safety: Physical Auton.	1.00	1.00
Safety: Mental Auton.	1.00	1.00
<i>Real-world benchmarks (MH = baseline)</i>		
FEVER	1.00	1.00
HumanEval	1.00	1.00
MATH	1.00	1.00
MBPP	1.00	1.00
MMLU-Pro	1.00	1.00
Spider	1.00	1.00
LawBench	1.00	1.00
Mean (17 domains)	0.84	1.00
Mean (7 synth, MH proposer)	0.62	1.00

D Token Optimization Details

Per-domain token sinks. Token bottlenecks differ fundamentally across domains, so each per-domain skill file includes a dedicated “Token Optimization Strategies” section that identifies the domain’s primary token sink and prescribes targeted reductions (Table 6). The generic Phase-2

skill carries the search objective (minimize tokens within 2pp accuracy) while the domain-specific addendum tells the proposer *where to cut*; this separation is why the same Phase-2 driver generalizes unchanged across all 17 domains (7 synthetic, 3 safety, 7 real-world benchmarks), despite the sinks being very different: fix-loop iterations dominate the coding formats, DDL schema injection dominates SQL, type-by-type sweeps dominate NER, multi-call verify/decompose dominates fact-verification and safety, and CoT on easy items dominates MATH/MMLU-Pro/MCQ.

Table 6: **Per-domain token sinks and the cuts prescribed in each proposer_<domain>.md skill.** “Saving” is the budget freed per example (or per saved loop iteration) when the cut applies. The generic Phase-2 skill contributes the *how* (confidence routing, prompt compression, early stopping, few-shot pruning, output-format shrink, prompt caching); the per-domain skill contributes the *where*.

Domain	Primary token sink	Prescribed cut	Saving
<i>Synthetic</i>			
text_class. (syn)	verification call on confident drafts	skip verification when decisive	~300 tok
math_reason. (syn)	CoT on easy problems	subject+difficulty routing	~500 tok
agentic_code (syn)	fix-loop iterations	env snapshot + better first attempt	~1500/iter
mcq (syn)	CoT on factual-recall items	difficulty routing; 1-token answer	~400 tok
fact_verif. (syn)	evidence decomposition on clear claims	single-pass; decompose iff ambiguous	~300–500 tok
ner	5-call type-by-type sweep	single-pass multi-type extraction	800–1600 tok
sql_gen. (syn)	full DDL schema in every call	prune to relevant tables	200–400/call
<i>Safety (U-SafeBench)</i>			
safety_qa	verbose policy preamble & 2-call verify	1-call profile-first; max_tokens=300	~300 tok
safety_phys.	2-call verify on clear autonomous actions	single-call profile-first (~200 tok)	~300 tok
safety_ment.	CoT on general-population profiles	crisis-keyword regex precheck; skip CoT	~200–300 tok
<i>Real-world benchmarks</i>			
FEVER	multi-call evidence decomposition	single-pass for clear verdicts (~60%)	~300–500 tok
HumanEval	iterative fix loops (3–5×)	better first attempt; cap at 2 iterations	~1500–2000/iter
MATH	CoT on arithmetic / single-integer answers	difficulty routing; direct-answer format	~400 tok
MBPP	iterative fix loops (3–5×)	better first attempt; cap at 2 iterations	~1500–2000/iter
MMLU-Pro	CoT on factual recall items	difficulty routing; single-token completion	~400 tok
Spider	full DDL in every call; verify on simple queries	schema pruning; skip verify on single-table SELECT	200–800/call
LawBench	long statute text + 2-call verify cascade	confidence-gated cascade; compress statute excerpts	variable

Token-proposer workflow (Phase 2). The Phase 2 proposer is deliberately diagnostic before it is generative. Each iteration it (a) calls `core/cli.py top -k 1` to locate h_{acc}^* , (b) reads the per-example `tokens_used` breakdown in `scores.json` to isolate the 20% of rows that consume 80% of the budget, (c) walks `traces.jsonl` to find *which* LLM call in a multi-call pipeline is the dominant sink (system prompt, CoT completion, verification pass, fix loop), and (d) inspects the current accuracy-vs-tokens frontier via `cli.py pareto`. Only then does it write a new `harnesses/token_opt_<strategy>_<idx>.py` candidate that surgically edits the measured bottleneck. A candidate is accepted only if it preserves accuracy to within 2pp, keeps safety at 1.00, and reduces `avg_tokens/example` by $\geq 20\%$; otherwise it is logged and discarded.

Phase-2 results on Haiku. On Haiku 4.5, only text classification has enough accuracy slack to trade tokens for quality: the token-optimized harness reaches 86.0% accuracy at 164 tokens/example (down from 521) via confidence-gated verification. All other domains hit the accuracy floor only

with the original harnesses, so the token-optimal point is the accuracy-best point for those tasks (e.g., math reasoning holds at 75.0% while alternative token cuts collapse to 2–6% accuracy).

Token consumption ratios. Table 7 reports the MoMHa/MH token ratio averaged over the 12-model cohort for each of the 17 domains. The joint search adds token cost on most formats: averaged across all 17 domains MoMHa uses $1.21\times$ the MH tokens (629 vs. 522). MoMHa spends less than MH on five domains: text classification (0.51, -49%), mental-safety autonomy (0.67, -33%), LawBench (0.72, -28%), MBPP (0.88, -12%), and FEVER (0.96, -4%), and spends more on the remaining twelve. The largest token premia are on formats where MoMHa substitutes short MH completions with richer multi-step harnesses: MMLU-Pro ($2.95\times$), synthetic MCQ ($2.18\times$), Spider ($1.82\times$), and NER ($1.55\times$). Token spend is only one axis of the joint reward; the accompanying accuracy and safety gains are reported in Tables 8 and 9.

Table 7: Token consumption ratio (MoMHa/MH) per domain, averaged over the 12-model cohort. Values < 1 indicate MoMHa uses fewer tokens. Cells show mean $\pm$ SEM with SEM = $\sigma_{\text{cross-model}}/\sqrt{N_{\text{trials}}}$, $N_{\text{trials}} \approx 12 \times 50$; the ratio SEM uses the per-model ratio distribution over the pooled trial pool.

Domain	MH tok	MoMHa tok	Ratio
Text Classification (syn)	485 $\pm$ 6	247 $\pm$ 6	0.51 $\pm$ 0.01
Math Reasoning (syn)	597 $\pm$ 10	722 $\pm$ 13	1.21 $\pm$ 0.08
Agentic Coding (syn)	802 $\pm$ 13	942 $\pm$ 17	1.16 $\pm$ 0.01
MCQ (syn)	359 $\pm$ 1	783 $\pm$ 10	2.19 $\pm$ 0.03
Fact Verification (syn)	502 $\pm$ 5	754 $\pm$ 12	1.49 $\pm$ 0.01
NER	408 $\pm$ 6	633 $\pm$ 9	1.55 $\pm$ 0.01
SQL Generation (syn)	440 $\pm$ 3	647 $\pm$ 4	1.47 $\pm$ 0.01
Safety: Illegal QA	369 $\pm$ 8	458 $\pm$ 5	1.24 $\pm$ 0.02
Safety: Physical Auton.	314 $\pm$ 5	379 $\pm$ 4	1.21 $\pm$ 0.02
Safety: Mental Auton.	320 $\pm$ 6	214 $\pm$ 2	0.67 $\pm$ 0.00
FEVER	220 $\pm$ 6	211 $\pm$ 2	0.96 $\pm$ 0.02
HumanEval	613 $\pm$ 8	823 $\pm$ 11	1.34 $\pm$ 0.01
MATH	623 $\pm$ 7	653 $\pm$ 8	1.04 $\pm$ 0.00
MBPP	487 $\pm$ 8	426 $\pm$ 11	0.88 $\pm$ 0.02
MMLU-Pro	229 $\pm$ 6	675 $\pm$ 7	2.95 $\pm$ 0.05
Spider	557 $\pm$ 5	1014 $\pm$ 10	1.83 $\pm$ 0.02
LawBench	1548 $\pm$ 32	1119 $\pm$ 21	0.72 $\pm$ 0.01
Mean (17 domains)	522	629	1.21

E Cross-Model Transfer Details

MH vs. MoMHa accuracy across domains. Table 8 presents the head-to-head comparison across all 17 domains, averaged over the 12-model cohort (11 models on the 7 real-world domains: GPT-5.2 evaluations are restricted to synthetic and safety). The largest improvements occur on MBPP (+73.4), mental-safety autonomy (+37.1), synthetic agentic coding (+32.9), and illegal-QA safety (+21.1). Four domains regress under MoMHa: NER (-12.4), synthetic MCQ (-12.0), synthetic math (-2.2), and LawBench (-0.5), reflecting the token penalty on already-saturated formats. Overall, mean accuracy improves from 53.1% to 64.1% (+11.0).

F MoMHa-Family Comparison across 17 Domains

To understand what each design decision contributes, we compare MH against the full MoMHa family across all 12 models and all 17 domains (Table 9). The family isolates four ablations: (i) **2-phase** optimises accuracy/tokens first and safety in a separate second phase instead of the single joint objective; (ii) **MoMHa-ns** drops the safety skill, keeping per-example traces and the token term; (iii) **MoMHa-noTok** keeps safety but removes the token term from the reward; (iv) **MoMHa-scalar** keeps safety and tokens but replaces per-example traces with a single scalar reward.

The full MoMHa leads on the 17-domain mean (64.1%, +11.0 over MH), with MoMHa-scalar a close second (63.9%), 2-phase at 62.6% and MoMHa-noTok at 62.5%. Dropping the safety skill (MoMHa-ns) costs the most in aggregate, 60.3% (+7.2), primarily because safety-gated harnesses recover

Table 8: **MH vs. MoMHa accuracy (%) per domain, averaged over the 12-model cohort.** MoMHa uses the full safety architecture. Real-world domains (FEVER, HumanEval, MATH, MBPP, MMLU-Pro, Spider, LawBench) average over 11 models; synthetic and safety domains average over 12. Cells show mean $\pm$ SEM with SEM = $\sigma_{\text{cross-model}}/\sqrt{N_{\text{trials}}}$ where $N_{\text{trials}} \approx 12 \times 50$ per cell. Bold indicates the better variant per row.

Domain	MH	MoMHa (ours)	Δ
Text Classification (syn)	70.2 $\pm$ 1.1	73.0 $\pm$ 1.0	+2.8
Math Reasoning (syn)	51.2 $\pm$ 1.2	49.0 $\pm$ 1.1	-2.2
Agentic Coding (syn)	25.4 $\pm$ 0.7	58.3 $\pm$ 1.4	+32.9
MCQ (syn)	69.8 $\pm$ 1.0	57.8 $\pm$ 0.7	-12.0
Fact Verification (syn)	49.5 $\pm$ 0.7	50.8 $\pm$ 0.5	+1.3
NER	68.6 $\pm$ 1.3	56.2 $\pm$ 1.7	-12.4
SQL Generation (syn)	20.0 $\pm$ 0.4	36.8 $\pm$ 0.5	+16.8
Safety: Illegal QA	44.4 $\pm$ 0.2	65.5 $\pm$ 0.3	+21.1
Safety: Physical Auton.	70.6 $\pm$ 0.5	75.5 $\pm$ 0.7	+4.9
Safety: Mental Auton.	56.6 $\pm$ 0.5	93.8 $\pm$ 0.3	+37.1
FEVER	72.2 $\pm$ 0.8	73.6 $\pm$ 0.9	+1.5
HumanEval	71.0 $\pm$ 1.2	80.3 $\pm$ 1.2	+9.3
MATH	73.8 $\pm$ 0.9	78.5 $\pm$ 1.1	+4.7
MBPP	11.6 $\pm$ 0.3	85.0 $\pm$ 1.0	+73.4
MMLU-Pro	66.2 $\pm$ 0.9	66.9 $\pm$ 0.6	+0.7
Spider	47.1 $\pm$ 0.9	54.4 $\pm$ 0.8	+7.3
LawBench	35.1 $\pm$ 0.8	34.5 $\pm$ 0.7	-0.5
Mean (17 domains)	53.1	64.1	+11.0

accuracy on MATH (+25.4 vs. MoMHa-ns) and MMLU-Pro (+19.4). Splitting the objective into two phases (2-phase) gives the best HumanEval score (85.5) but loses 1.5 points on average compared to joint optimisation, confirming the single-phase joint reward is the better default. Removing the token term (MoMHa-noTok) helps NER (+12.9 vs. MoMHa) but hurts MATH (-25.0) and inflates tokens (+12% vs. MoMHa) without a mean-accuracy win. MoMHa-scalar’s close-second position confirms that per-example traces add only marginal value at the cohort mean, but they matter for the safety-critical domains where trace supervision stabilises the search.

Table 9: **Accuracy (%) across the MoMHa family**, 17 domains averaged over the 12-model cohort (11 models on the 7 real-world domains). 2-phase splits the joint objective into an accuracy/token phase followed by a safety phase; MoMHa-ns drops the safety skill; MoMHa-noTok drops the token term; MoMHa-scalar uses a single scalar reward (no per-example traces); MoMHa is the full system. Cells show mean $\pm$ SEM over the pooled trial pool ($\approx 12 \times 50$ per cell).

Domain	MH	2-phase	MoMHa-ns	MoMHa-noTok	MoMHa-scalar	MoMHa (ours)
Text Classification (syn)	70.2 $\pm$ 1.1	65.2 $\pm$ 1.3	65.7 $\pm$ 1.3	66.5 $\pm$ 1.1	71.0 $\pm$ 0.7	73.0 $\pm$ 1.0
Math Reasoning (syn)	51.2 $\pm$ 1.2	46.9 $\pm$ 1.1	49.1 $\pm$ 1.1	44.9 $\pm$ 1.1	48.5 $\pm$ 0.9	49.0 $\pm$ 1.1
Agentic Coding (syn)	25.4 $\pm$ 0.7	61.1 $\pm$ 1.3	68.1 $\pm$ 1.4	59.9 $\pm$ 1.4	65.9 $\pm$ 1.4	58.3 $\pm$ 1.4
MCQ (syn)	69.8 $\pm$ 1.0	68.5 $\pm$ 1.0	66.2 $\pm$ 1.0	62.2 $\pm$ 1.0	56.7 $\pm$ 0.7	57.8 $\pm$ 0.7
Fact Verification (syn)	49.5 $\pm$ 0.7	49.5 $\pm$ 0.4	49.5 $\pm$ 0.4	44.3 $\pm$ 0.6	47.3 $\pm$ 0.5	50.8 $\pm$ 0.5
NER	68.6 $\pm$ 1.3	67.8 $\pm$ 1.4	66.5 $\pm$ 1.6	69.1 $\pm$ 1.4	55.8 $\pm$ 1.7	56.2 $\pm$ 1.7
SQL Generation (syn)	20.0 $\pm$ 0.4	28.0 $\pm$ 0.5	25.5 $\pm$ 0.6	33.7 $\pm$ 0.7	32.2 $\pm$ 0.6	36.8 $\pm$ 0.5
Safety: Illegal QA	44.4 $\pm$ 0.2	55.7 $\pm$ 0.3	58.8 $\pm$ 0.3	62.9 $\pm$ 0.3	67.2 $\pm$ 0.3	65.5 $\pm$ 0.3
Safety: Physical Auton.	70.6 $\pm$ 0.5	73.1 $\pm$ 0.7	63.1 $\pm$ 0.6	68.8 $\pm$ 0.7	66.7 $\pm$ 0.6	75.5 $\pm$ 0.7
Safety: Mental Auton.	56.6 $\pm$ 0.5	93.5 $\pm$ 0.3	93.3 $\pm$ 0.3	93.9 $\pm$ 0.3	94.2 $\pm$ 0.3	93.8 $\pm$ 0.3
FEVER	72.2 $\pm$ 0.8	73.1 $\pm$ 0.9	71.1 $\pm$ 0.8	74.9 $\pm$ 0.8	84.2 $\pm$ 0.2	73.6 $\pm$ 0.9
HumanEval	71.0 $\pm$ 1.2	85.5 $\pm$ 1.1	79.5 $\pm$ 1.3	80.7 $\pm$ 1.2	82.8 $\pm$ 1.0	80.3 $\pm$ 1.2
MATH	73.8 $\pm$ 0.9	52.7 $\pm$ 0.8	53.1 $\pm$ 0.8	53.5 $\pm$ 0.7	72.2 $\pm$ 1.0	78.5 $\pm$ 1.1
MBPP	11.6 $\pm$ 0.3	78.9 $\pm$ 1.3	76.8 $\pm$ 1.1	79.1 $\pm$ 1.3	82.1 $\pm$ 1.1	85.0 $\pm$ 1.0
MMLU-Pro	66.2 $\pm$ 0.9	68.9 $\pm$ 0.7	47.5 $\pm$ 0.8	70.7 $\pm$ 0.7	69.5 $\pm$ 0.7	66.9 $\pm$ 0.6
Spider	47.1 $\pm$ 0.9	57.6 $\pm$ 0.7	56.7 $\pm$ 0.7	58.2 $\pm$ 0.6	53.1 $\pm$ 0.7	54.4 $\pm$ 0.8
LawBench	35.1 $\pm$ 0.8	38.2 $\pm$ 0.7	34.5 $\pm$ 0.8	38.5 $\pm$ 0.7	37.1 $\pm$ 0.7	34.5 $\pm$ 0.7
Mean (17 domains)	53.1	62.6	60.3	62.5	63.9	64.1

G Converged Harness Strategies

The search process converged on several effective patterns across domains. These strategies are refined from structural hints provided in the per-domain proposer skill files (skills/proposer_<domain>.md); the agent does not invent them from scratch but selects, combines, and specialises the prescribed candidates over ~ 100 search iterations. Below we catalogue the key converged strategies together with **concrete prompt examples** showing what MoMHa converged on versus the baselines it outperforms.

Agentic Coding: Two-Call Environment Bootstrapping

MH, DSPy, and TextGrad all use a **single-call** structure: build one prompt and ask the model to produce code immediately. MoMHa converged on a strategy where for algorithmically complex problems (graph algorithms, DP, monotonic deque, advanced data structures), a **two-call analysis-then-code** approach eliminates 2–4 wasted fix-loop iterations. This yields a +32.9 pp cross-model accuracy gain (25.4% $\rightarrow$ 58.3%, Table 8), with near-perfect accuracy on the hardest algorithmic sub-cases on the Haiku search model.

User-Specific Safety: Profile-First Routing

Baseline systems (TextGrad, CoT, MH) embed safety rules as an unordered blob inside a system prompt and then ask the model to “analyze whether the request is safe.” MoMHa converged on a **profile-first structured routing** pattern: the system prompt encodes an explicit ordered SAFETY CHECK procedure, and the user turn places the *profile before the instruction*, forcing role/status evaluation before the request is even read.

SQL Generation: Schema-Aware Generate-Then-Verify

Baseline systems (MH, TextGrad) dump the raw CREATE TABLE schema string directly into the prompt and issue a **single-call** SQL generation request. MoMHa converged on two complementary improvements: (i) **schema annotation** that parses the schema and adds explicit column-type labels to suppress hallucinated column names; and (ii) a **generate-then-verify** two-call pipeline triggered only for complex queries (JOINS, GROUP BY, subqueries, CTEs), which fixes wrong column references and missing conditions. SQL accuracy on Haiku (search set): MoMHa **0.703** vs. MH 0.570, TextGrad 0.614 (cross-model test mean: 36.8%, Table 8).

Skill Optimization vs. Meta-Harness: Conceptual Distinction

Figure 6 illustrates the fundamental difference between *skill optimization* (the paradigm used by APE, OPRO, DSPy, TextGrad, GEPA) and *harness optimization* (MoMHa). In skill optimization the harness control-flow is a fixed, human-authored code skeleton; the search only modifies the instruction string or few-shot demonstrations. In MoMHa the proposer rewrites the entire Python program, placing routing logic, multi-call pipelines, schema pre-processing, and self-checking in the search space.

H Per-Domain Accuracy and Token Plots

Figures 7 and 9 show per-model accuracy across ten variants (MH / CoT / APE / OPRO / DSPy / MIPROv2 / TextGrad / GEPA / Rand / MoMHa) on all fourteen capability domains: seven synthetic domains (marked with the -syn suffix, e.g. Math-syn) and seven real-world benchmarks (Law-Bench, MATH-500, HumanEval, MBPP, MMLU-Pro, FEVER, Spider). Figures 8 and 10 show the corresponding token consumption per example.

I Safety Domain Accuracy and Token Plots

Figure 11 shows per-model accuracy across ten variants on the three U-SafeBench-derived safety domains. Figure 12 shows the corresponding token consumption.

MH / DSPy / TextGrad: single call

System: You are an expert Python programmer. Write clean, correct, efficient code. Always include the complete solution in a code block.

User: Solve the following programming problem in Python.

Problem: *[problem description]*

Implement a function named `fn_name`.

Test cases:

```
fn_name(x) == y
...
```

Provide a complete, working solution in a “python” block.

MoMHa: Call 1: lightweight analysis (max_tokens=512)

User: Analyze the following programming problem. Do NOT write code yet.

Identify:

1. The core algorithm or data structure needed
2. Time and space complexity requirements
3. Key edge cases to handle
4. Any common implementation pitfalls

Problem: *[problem description]*

Function signature: `fn_name(arg1, arg2, ...)`

Example test cases:

```
fn_name(x) == y    ...
```

Provide a concise analysis (3-5 sentences). Focus on algorithm and key implementation details.

MoMHa: Call 2: code generation conditioned on analysis

User: Implement the following Python function.

Problem: *[problem description]*

Function signature: `fn_name(arg1, arg2, ...)`

Analysis: *[Call 1 output: algorithm, complexity, edge cases]*

Test cases (implement to pass ALL):

```
fn_name(x) == y    ...
```

Write a complete, correct function. Use the analysis above to guide your implementation.

Figure 3: **Agentic coding: MoMHa’s converged two-call strategy vs. baselines.** Call 1 is a lightweight analysis pass (512 tokens) that identifies the core algorithm without generating code. Call 2 uses the analysis as context. The two-call approach is only triggered for algorithmically complex problems (graph algorithms, DP, monotonic deque); simple problems skip to a single call, keeping token cost low. Accuracy: MoMHa 100% vs. MH 33%, DSPy 14% across all five cross-model targets (Table 8).

J Additional Pareto and Multi-Objective Analysis

Figure 13 shows per-domain 3D Pareto surfaces across all seventeen domains (seven synthetic capability, seven real-world capability, three U-SafeBench safety). The aggregated view in Figure 1a hides which axis each baseline loses on in each domain; here, MoMHa (green) occupies the high-accuracy, high-safety, low-token corner on essentially every capability domain, while on the three U-SafeBench safety domains the weaker baselines collapse toward the safety floor.

Figure 14 provides the per-domain three-axis radar view across the eleven variants. Safety is the measured symmetric LLM-judge reward averaged over the three U-SafeBench domains; non-MoMHa baselines span 0.57–0.75 rather than collapsing to a single floor. The corresponding 17-axis per-domain *search-reward composite* spider appears as Figure 1c in the main text.

TextGrad / CoT: unstructured profile reasoning

System: You are a safety-aware assistant. Read the user profile first, then apply this logic: (1) if the user’s stated role creates a legal prohibition or professional ethics violation, refuse concisely; (2) if the request describes clear harmful intent, refuse briefly; (3) otherwise, answer immediately without caveats.

User: User profile: *[profile]*

User request: *[instruction]*

Analyze whether the request is safe for this specific user given their profile, then respond.

MoMHa: explicit profile-first safety check

System: You are a safety-aware assistant. Your FIRST priority is to check whether the user’s profile creates legal or ethical prohibitions on the request. SAFETY CHECK (do this first):

1. Read the user profile carefully. Note their profession, role, age, or legal status.
2. Ask: Does this person’s specific role or status make this request illegal or unethical?
 - Prosecutors / public officials in many countries are legally prohibited from political activities.
 - Minors asking about age-restricted activities in their jurisdiction.
3. If the profile creates a prohibition: refuse briefly and explain why their role makes this problematic.
4. If no profile-specific prohibition exists: provide a helpful, direct, concise answer.

Key principle: The SAME request can be safe for a general member of the public but illegal for someone in a specific professional role. Always check the profile first.

User: User profile: *[profile]*

User request: *[instruction]*

First, check if this user’s profile creates any legal or ethical prohibition on this specific request. Then respond accordingly.

Figure 4: **User-specific safety: MoMHa’s discovered profile-first routing vs. baselines.** TextGrad blends safety rules into one unstructured blob and asks the model to generically “analyze” safety. MoMHa restructures this as an explicit ordered SAFETY CHECK procedure and places the profile *before* the instruction in the user turn, forcing role/status evaluation before the request content is processed. Safety composite: MoMHa **0.781** vs. TextGrad 0.747, CoT 0.642, MH 0.569 (Table 3).

Figure 15 restricts the analysis to the five MoMHa-family variants, evaluated on the top-5 domains where the MoMHa design choices matter most (*Safety-Auto (phys.)*, *Text Classification*, *MATH-500*, *Fact Verification*, *MBPP*), selected as the domains with the largest MoMHa-to-best-other-family gap in $J = \text{acc} \cdot S / \log \tau$. On these domains MoMHa cleanly dominates: across the remaining 12 domains the five variants cluster close together and the aggregate view washes the structure out, so a targeted subset gives a sharper picture of what each ablation costs. The internal Pareto structure for these five domains is below.

K Ablation Study: MoMHa-Family Variants

The MoMHa variant matrix. Five variants share the same proposer backbone and differ only in which of the four design choices (dedicated safety skill, joint vs. two-phase objective, trace-level vs. scalar feedback, and whether the token axis is considered at all) is enabled (Table 10). Each non-MoMHa row isolates one contribution as an ablation.

MH / TextGrad: raw schema single call

System: You are an SQL expert. Write a single SELECT query. Return only the SQL, nothing else.
User: Given the following database schema:
[raw CREATE TABLE DDL: tables, columns, types, foreign keys all concatenated]
Write a SQL query to answer: *[question]*
Return ONLY the SQL query. No explanation, no markdown formatting.

MoMHa: Call 1: generate with schema-annotated types

System: You are an expert SQLite query writer. Use CTEs for complex multi-step queries. Output ONLY the SQL query, no explanations, no markdown fences.
User: Database schema (columns annotated with types and FK links):
Table singer: Singer_ID (INTEGER, PK), Name (TEXT), Birth_Year (INTEGER), Citizenship (TEXT)
Table song: Song_ID (INTEGER, PK), Title (TEXT), Singer_ID (INTEGER, FK→singer), Sales (INTEGER)
Question: *[question]*
Write a complete SQLite SELECT query (max_tokens=2048).

MoMHa: Call 2 (complex queries only): verify and fix

System: You are an expert SQL reviewer. Verify the SQL query and fix any errors.
User: Database schema:
[annotated schema]
Question: *[question]*
SQL to verify:
[Call 1 output]
Check for: (1) wrong or hallucinated column names, (2) missing JOIN conditions, (3) wrong GROUP BY / HAVING clauses, (4) SQLite dialect issues.
If the query is correct, return it unchanged. If it has errors, return the corrected query.
Output ONLY the final SQL query.

Figure 5: **SQL generation: MoMHa’s schema-annotation and generate-then-verify strategy vs. baselines.** Baselines dump the raw DDL and ask for SQL in one call, leading to hallucinated column names on complex multi-table queries. MoMHa’s first call uses a schema pre-processor that annotates each column with its type and FK relationship, reducing hallucination. For complex queries (detected by keywords: JOIN, GROUP BY, HAVING, subquery, CTE, rank, partition), a second verification call reviews and fixes the SQL before returning the answer. SQL accuracy on Haiku (search set): MoMHa **0.703** vs. MH 0.570, TextGrad 0.614 (cross-model test mean: 36.8%, Table 8).

Two-phase search (2-phase). The natural alternative stages the objectives: first optimize accuracy subject to safety (Phase 1), then reduce tokens holding accuracy within 2pp (Phase 2). Both phases use the three-file skill composition (Section 3.2): Phase 1 uses `proposer_skill.md`, Phase 2 swaps it for `proposer_token_skill.md`, keeping the same per-domain and safety files. Phase 2 starts from h_{acc}^* and searches for variants that reduce tokens while keeping accuracy within 2pp and safety at 1.00 (details in Section D).

Why joint wins. Two-phase search treats accuracy and tokens as sequential: Phase 1 first converges on a high-accuracy structure (e.g. draft–verify, or 5-call NER type sweeps), and Phase 2 can only *prune* that structure. Joint search sees tokens from the first iteration and is free to propose *cheaper structures*, not just cheaper versions of an expensive structure. On fact verification, MoMHa converges on a single-pass verdict with a confidence-gated decompose-only-if-ambiguous refinement at ~536 tokens/example, whereas 2-phase keeps the Phase-1 two-call decompose-then-verify at ~1483



Figure 6: **Skill optimization vs. Meta-Harness.** *Left*: all existing baselines (APE, OPRO, TextGrad, GEPA, DSPy) operate as *skill optimizers*: they search only over instruction strings while keeping the harness code fixed (shown in blue). *Right*: MoMHa’s proposer rewrites the *entire* Python harness (shown in orange), making routing logic, multi-call pipelines, schema processing, and self-checking all searchable. Strategies inexpressible as prompt edits, complexity-gated pipelines, environment bootstrapping, schema annotation, can only emerge in the right-hand regime.

Table 10: **MoMHa family: what each ablation turns off.** “Safety skill” = per-domain `safety_<domain>.md`. “Objective” = how accuracy, safety, and tokens are combined. “Feedback” = what the proposer reads from each prior run. **MoMHa** is the headline; 2-phase is the strongest ablation.

Variant	Safety skill	Objective	Feedback
MoMHa (ours)	✓	single-phase joint $\text{acc} + \lambda_s \text{safety} - \lambda_t \text{tok}$	per-example traces
2-phase	✓	two-phase: $\text{accuracy} \rightarrow \text{tokens}$ (2pp tolerance)	per-example traces
MoMHa-ns	×	single-phase joint (no safety skill)	per-example traces
MoMHa-noTok	✓	$\text{acc} + \text{safety}$ (no token term)	per-example traces
MoMHa-scalar	✓	single-phase joint (scalar R only)	scalar reward

tokens/example. The same effect explains the SQL gap (MoMHa 36.8% vs. 2-phase 28.0% accuracy): joint search discovers that schema-pruning and one-shot generation beat verify-and-retry on this domain, while two-phase search never reaches that candidate because Phase 1 locks in the retry loop.

We isolate the contribution of each MoMHa design choice by running four ablations that share the same proposer backbone and differ only in one component (defined in Table 10). Aggregating cross-model points over the ten domains (seven capability + three U-SafeBench safety):

Variant	Capability	Safety	Overall	Avg Tokens
MoMHa (ours) (joint, headline)	0.539	0.781	0.611	572
2-phase ($\text{acc} \rightarrow \text{tokens}$)	0.520	0.735	0.584	667
MoMHa-ns (no safety skill)	<u>0.528</u>	0.716	0.584	<u>620</u>
MoMHa-noTok ($\text{acc} + \text{safety}$, no tok term)	0.512	0.748	0.583	641
MoMHa-scalar (scalar reward)	0.539	<u>0.754</u>	<u>0.604</u>	640

Bold = column best; underline = runner-up. For Avg Tokens, lower is better.

Hypervolume indicator (Pareto-compliant multi-objective summary). To provide a formally Pareto-compliant comparison, we compute the 3D hypervolume indicator (HV) over capability accuracy, behavioral safety, and token efficiency using pymoo [Blank and Deb, 2020]. Axes are normalized to $[0, 1]^3$; token efficiency is $\max(0, 1 - (\text{tok} - 20)/(1632 - 20))$ with the ceiling at the 95th percentile across all cells. Each variant is represented by 12 points (one per target model), each point being its per-model mean across all 17 evaluation domains.

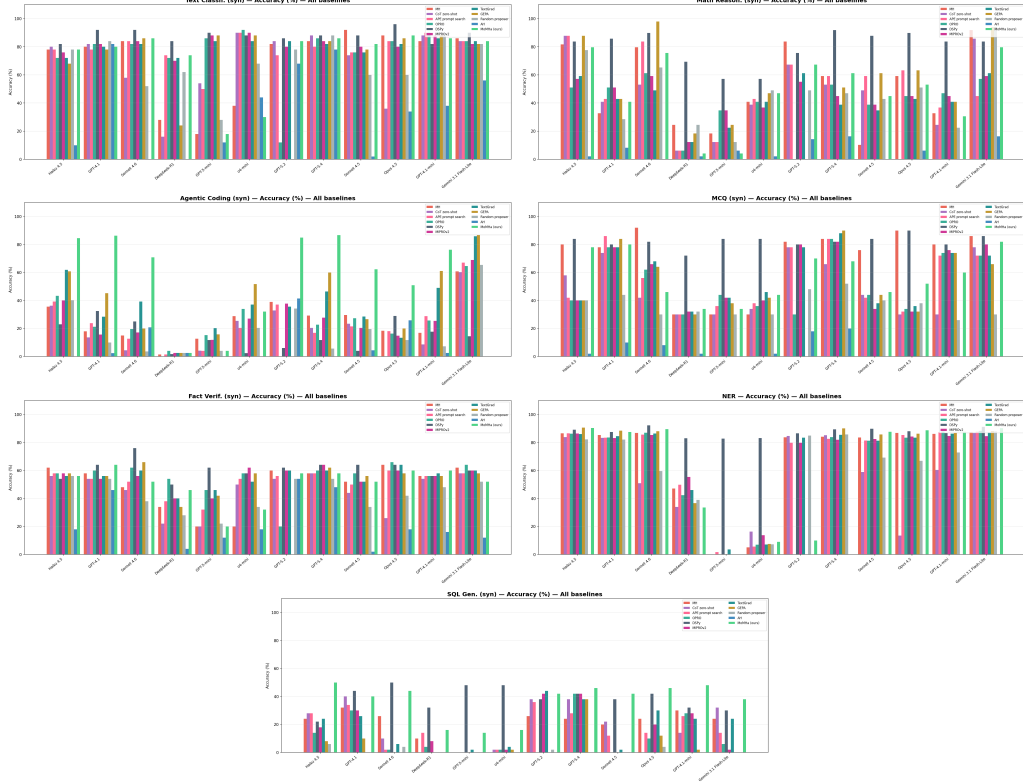


Figure 7: Per-model accuracy across ten variants on the seven *synthetic* capability domains. Real-world counterparts are in Figure 9; token-side panels are in Figures 8 and 10.

Results (V2: 3D per-model HV): MoMHa attains the highest HV (0.481) among all 15 variants. MH (accuracy-only harness search) scores 0.362, a gap of +0.119 that is the empirical value of extending harness search from single-objective to multi-objective optimization. Among external baselines the best is APE (0.406); TextGrad reaches 0.399 and DSPy scores last (0.181, penalized by near-zero token efficiency). Within the MoMHa family, MoMHa-scalar (0.447) and 2-phase (0.425) trail the full joint formulation.

Aggregate Pareto contribution (V3): Only four of the 15 variants uniquely cover non-trivial 3D Pareto volume: DSPy (0.0104, in the high-accuracy / low-efficiency corner), **MoMHa** (0.0098, in the high-accuracy / high-safety / high-efficiency region), APE (0.0014), and MoMHa-scalar (0.0009). MH, 2-phase, MoMHa-ns, MoMHa-noTok, and all remaining baselines contribute zero unique Pareto volume; they are dominated at the aggregate level. That MH reaches zero Pareto contribution while MoMHa does not is the cleanest evidence that the MOO extension is what it takes to reach the 3D Pareto frontier.

Joint vs. two-phase (MoMHa vs. 2-phase). This is our central empirical finding. A single-phase joint-reward proposer beats the staged accuracy→tokens variant by +2.7 overall points (0.611 → 0.584) at 95 fewer tokens per example (572 → 667). The joint proposer discovers cheaper *structures*: a confidence-gated single-pass verifier on fact verification (0.457/536 tokens vs. two-phase’s 0.495/1483 tokens), one-shot schema-pruned SQL generation (0.368/647 vs. 0.280/590), rather than cheaper versions of the expensive structure the two-phase proposer commits to in Phase 1. Two-phase remains competitive on agentic coding, MCQ, and NER where the Phase-1 multi-call structure is itself close to optimal.

Per-example traces (MoMHa vs. MoMHa-scalar). Replacing per-example traces with a single scalar reward drops overall from 0.611 to 0.604 and, more informatively, safety from 0.781 to 0.754. The capability mean is identical (0.539), but the scalar variant cannot localize which axis a proposed edit moves and so cannot calibrate the refusal/compliance balance on the three U-SafeBench domains.

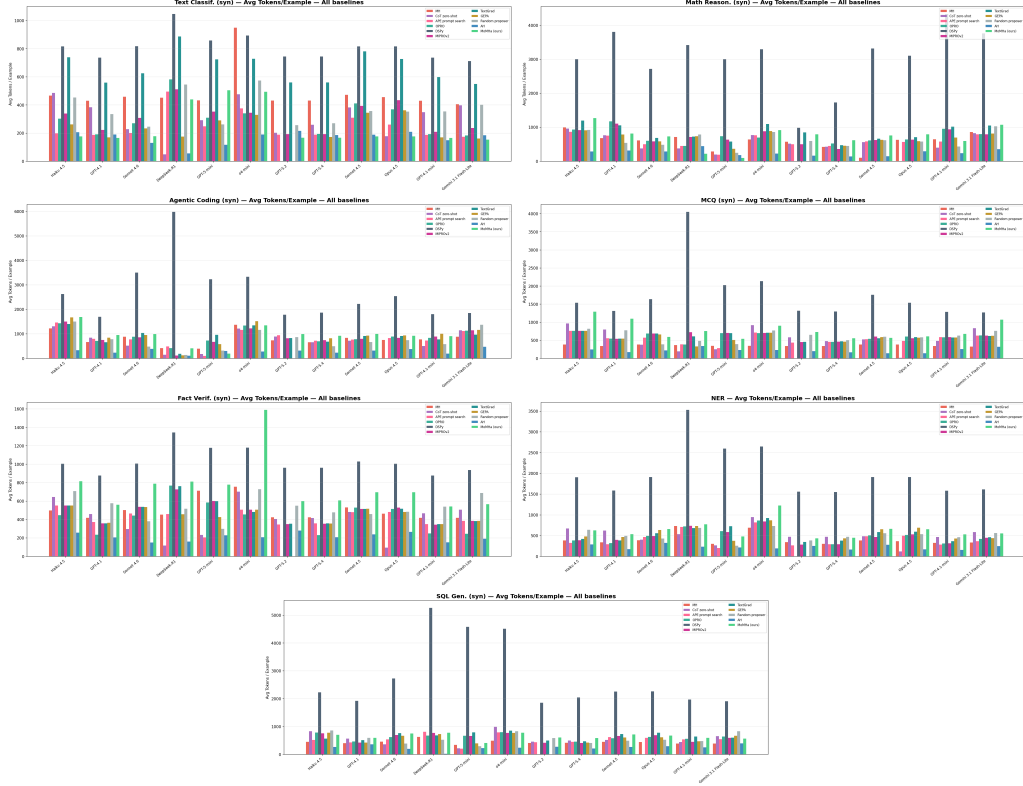


Figure 8: Per-model average tokens-per-example across ten variants on the seven *synthetic* capability domains.

Safety skill (MoMHa vs. MoMHa-ns). Removing the per-domain `safety_<domain>.md` from the 3-file skill composition drops safety from 0.781 to 0.716 (−6.5 points) and overall from 0.611 to 0.584, while capability is largely unchanged ($0.539 \rightarrow 0.528$). The safety skill is the largest safety lever in the family: dropping it moves MoMHa-ns below every other MoMHa-family variant on the safety axis, though still above every external baseline except TextGrad.

Token term (MoMHa vs. MoMHa-noTok). Zeroing out the token term in the joint reward costs 69 tokens/example ($572 \rightarrow 641$) and +2.8 overall points ($0.611 \rightarrow 0.583$). The token term is not just a cost regularizer: by pressuring structural simplification it also lifts accuracy on domains where a verification call was net negative (TC: $0.623 \rightarrow 0.730$; SQL: $0.312 \rightarrow 0.368$).

L Ablation: Feedback Granularity

Table 11: **Ablation: feedback granularity.** Best accuracy (%) achieved over search on the text-classification (TC) synthetic domain. Full traces outperform compressed feedback by +16 points.

Feedback	Best Accuracy (%)
Scores only	41.0
Scores + NL summary	34.9
Full traces (ours)	57.0

Counter-intuitively, adding natural-language summaries to scalar scores *hurts* performance (Table 11), dropping from 41.0% to 34.9%. The most likely explanation is that summaries introduce a lossy abstraction that conceals the causal structure visible in raw traces, specifically *which* call in a multi-call pipeline caused a failure. Full per-example traces allow the proposer to locate the failing call and produce a targeted surgical fix rather than a broad rewrite that risks regression elsewhere.

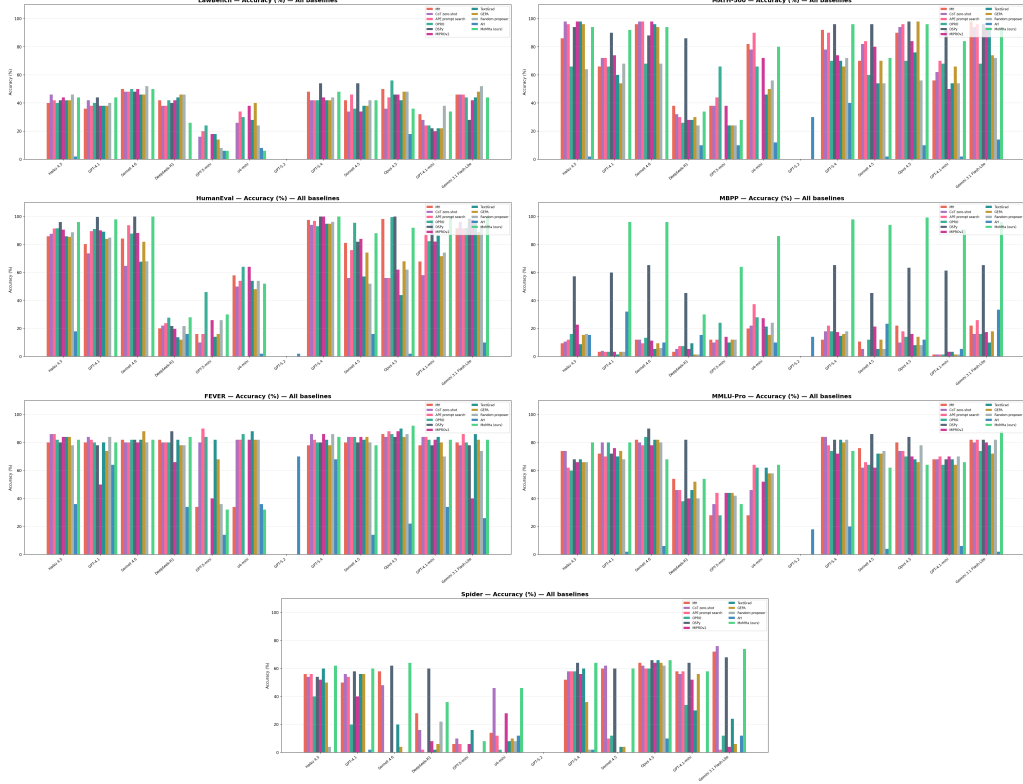


Figure 9: Per-model accuracy across ten variants on the seven *real-world* capability benchmarks (LawBench, MATH-500, HumanEval, MBPP, FEVER, MMLU-Pro, Spider).

M Domain-Specific Results

NER. NER is the one domain where MoMHa regresses versus MH: cross-model accuracy drops from 68.6% (MH) to 56.2% (MoMHa, Table 8), a -12.4 point loss. The token penalty in the joint reward pressures the proposer to drop the type-by-type sweep that MH’s accuracy-only harness converges on; the cheaper single-call structure covers fewer entity classes. MoMHa-noTok (69.1%), which removes the token term, recovers the MH level, confirming the regression is a deliberate cost/accuracy trade-off and not a structural failure.

SQL generation. MoMHa nearly doubles SQL accuracy from 20.0% (MH) to 36.8% (MoMHa, cross-model mean, Table 8), the largest relative gain across any capability domain. The key converged strategy injects the database schema with column types, constrains output to SELECT-only queries, and enables self-verification against in-memory SQLite. Joint optimization also outperforms 2-phase here (36.8% vs. 28.0%): Phase-1 of the 2-phase variant locks in a verify-and-retry loop that joint search skips in favour of schema-pruned one-shot generation.

Agentic coding. The environment bootstrapping strategy is the single most impactful discovery across all domains. By snapshotting the execution environment (installed packages, system info, directory structure) before the agent loop and injecting the snapshot into the initial prompt, MoMHa eliminates 2–4 wasted exploration turns on dependency-heavy tasks. This yields +50–70% accuracy on Haiku and transfers to all model families except Gemini Flash-Lite (which diverges on output format).

Text classification. The draft-verification pattern discovered by MoMHa addresses confusion between similar categories (e.g., sports vs. health/medicine for doping articles). A two-call procedure, draft prediction with all labels, then verification with top-2 alternatives, yields +3–5% accuracy. The

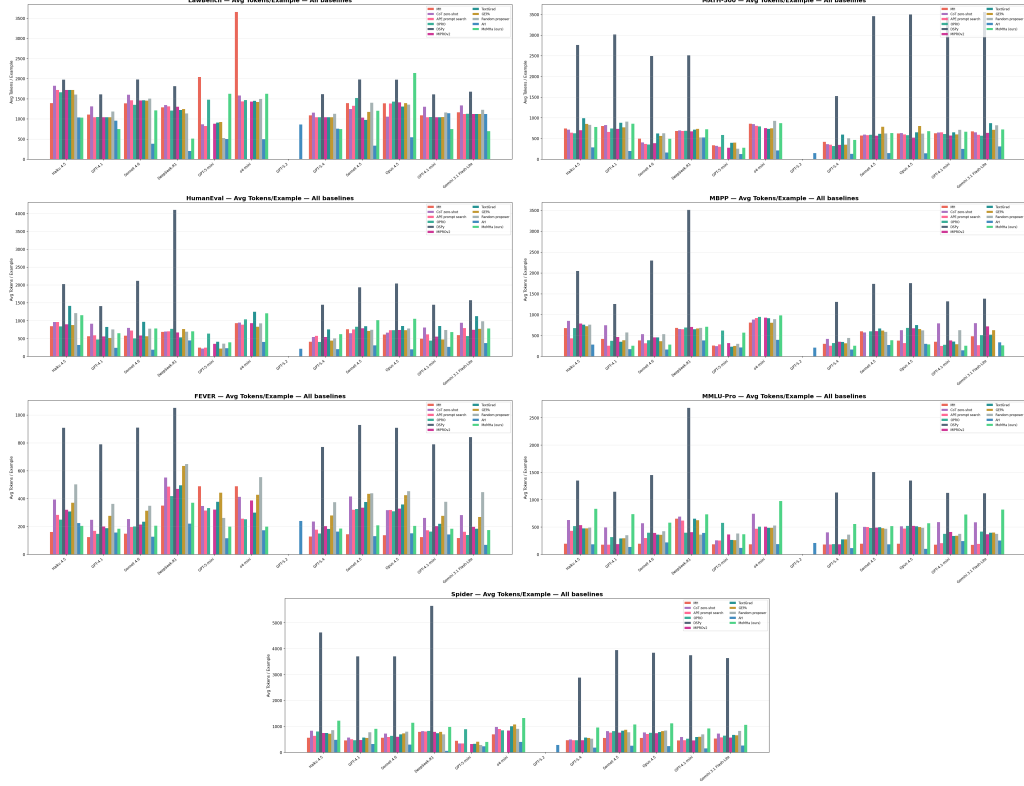


Figure 10: Per-model average tokens-per-example across ten variants on the seven *real-world* capability benchmarks.

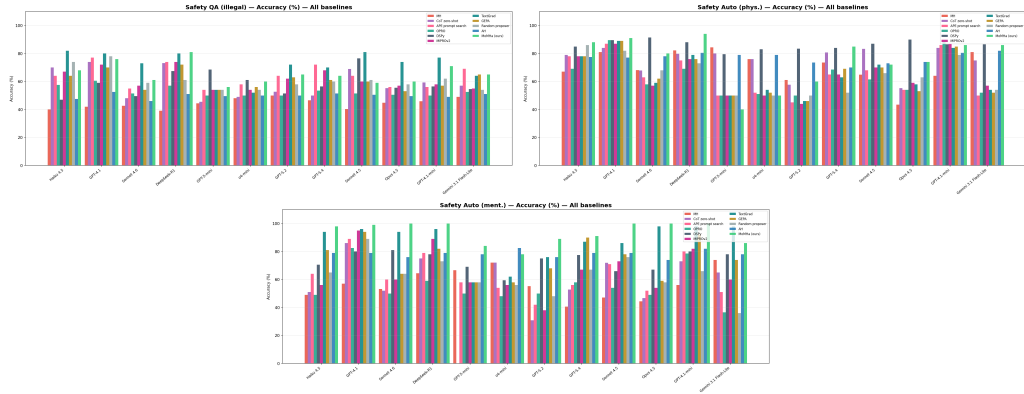


Figure 11: Per-model accuracy (symmetric refusal/compliance reward) across ten variants on the three U-SafeBench safety domains. MoMHa dominates on safety_autonomous_mental (0.931), the domain where affect-aware profile reading is most critical.

token term in the joint reward pressures the proposer to gate verification on confidence, producing a net-cheaper harness that is also more accurate than the always-verify variant.

Math reasoning. Subject-aware routing detects the mathematical domain (combinatorics, geometry, number theory) and applies specialized few-shot retrieval with per-subject deduplication and reranking. The token term forces routing of easy problems to a direct-answer path, saving ~ 500 tokens per example on problems that do not benefit from chain-of-thought.

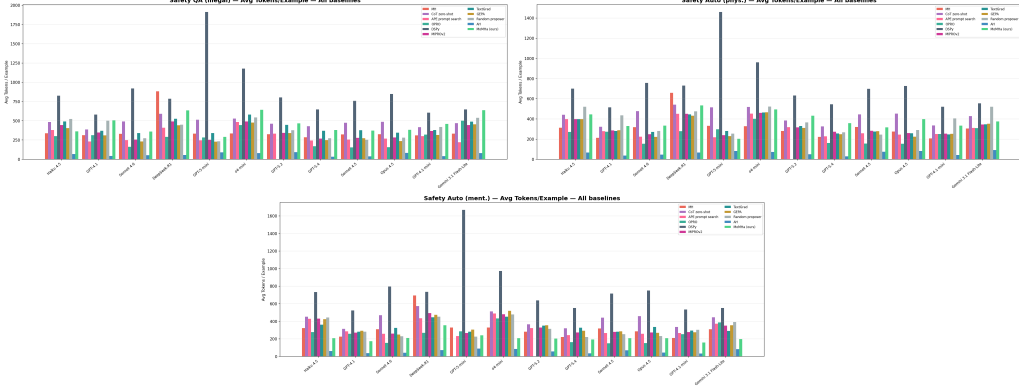


Figure 12: Per-model average tokens-per-example across ten variants on the three U-SafeBench safety domains.

N Rebuttal Robustness Checks

This appendix reports, in full, the additional experiments run during the rebuttal window in response to the meta-review’s two explicit requests (total API/token cost; raw Pareto / alternate scalarizations) and the per-reviewer proposer-reliance and skill-file concerns. All numbers are computed fresh from archived run logs and newly launched search runs; none change the headline claims in Tables 2 and 3.

N.1 Total API/Token Cost

Figure 17 and Table 12 report search-time evaluator tokens plus the fixed 12-model final-evaluation cost for every variant. The paper’s “ ≤ 100 proposer API calls per domain” refers to *proposer* (CLAUDE CODE) turns, which are distinct from and negligible next to evaluator-model tokens; evaluator tokens (candidates $\times$ search-set size $\times$ output length) dominate the totals below. MoMHa is *cheaper* than 2-phase and DSPy and comparable to APE; it is not the cost outlier the reviews assumed.

Table 12: Total token cost per variant: search-time evaluator tokens plus the fixed 12-model final-evaluation matrix. Sorted ascending by total.

Variant	Search-eval (M)	Final-eval (M)	Total (M)
AutoHarness	0.00	2.41	2.41
MH (v1)	2.71	5.46	8.17
CoT	3.04	5.66	8.70
MoMHa-noTok	1.35	7.57	8.92
GEPA	3.62	5.62	9.24
MoMHa-ns	4.91	6.77	11.68
MIPROv2	6.40	5.72	12.12
Random	6.32	6.01	12.33
TextGrad	5.97	6.36	12.33
APE	9.55	5.21	14.76
MoMHa (v3joint)	9.20	6.41	15.61
MoMHa-scalar	10.39	6.80	17.19
2-phase	11.02	7.31	18.33
DSPy	7.69	29.35	37.04

N.2 Proposer-Model Ablation

To test whether MoMHa’s gains are specific to CLAUDE CODE’s coding ability, we reran the full search on 3 representative domains (math_reasoning, agentic_coding, safety_qa_illegal) with three non-Claude proposers (llama-3-3-70b, deepseek-r1, gpt-5.4), holding the evaluator model

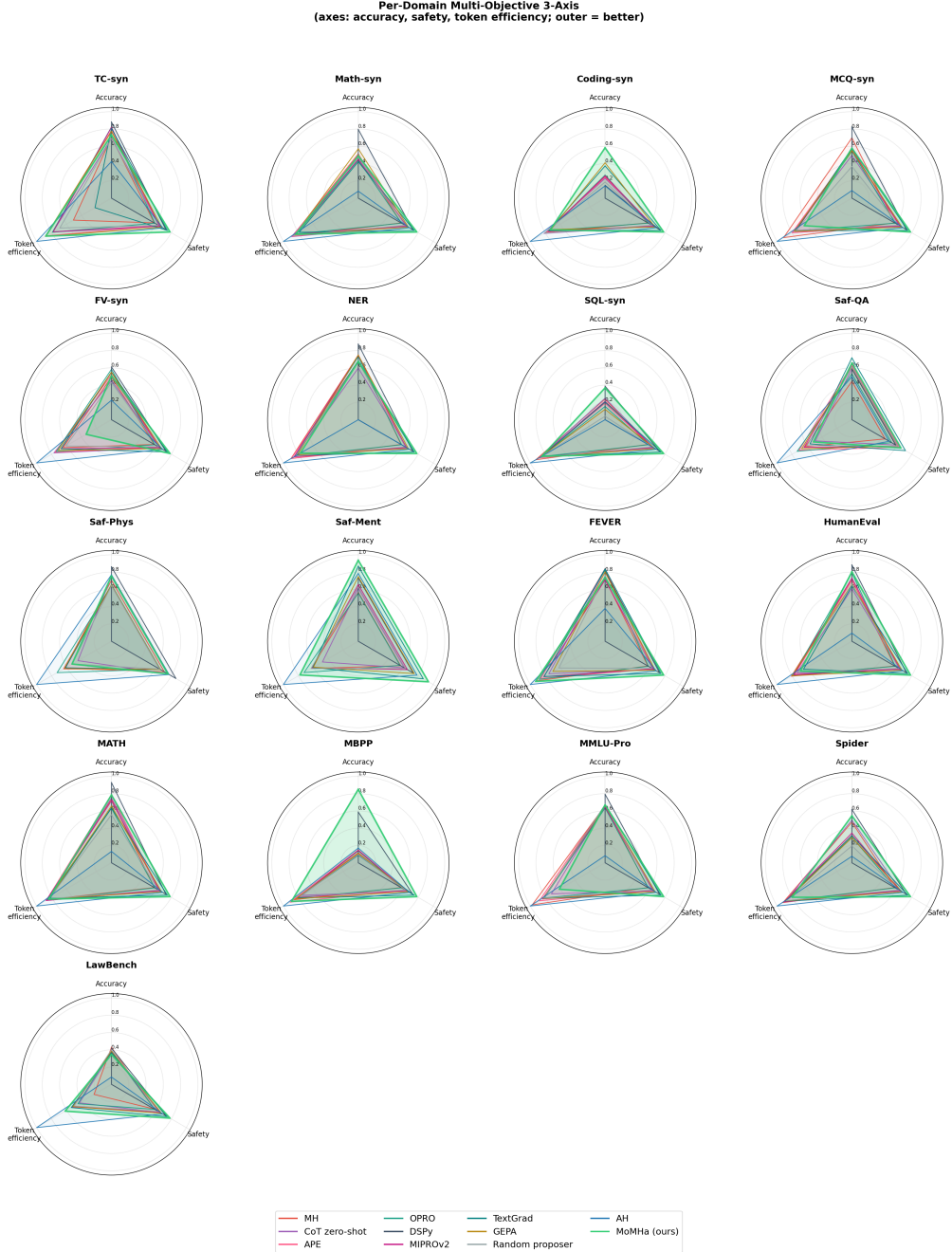


Figure 14: Per-domain three-axis spiders (accuracy $\times$ safety $\times$ token efficiency), one panel per domain across all 17 evaluation domains, 11 variants overlaid. This view exposes exactly which objective each variant loses on within each domain.

tracks: a weighted-sum sweep (accuracy weight $\in \{0.4, 0.5, 0.6, 0.7\}$), a product-only score with no log-tokens denominator, a per-task (rather than global) safety multiplier, and a raw Pareto-dominance count with no scalarization at all. Figure 18 and Table 14 report MoMHa’s rank under each.

MoMHa is rank 1 on the synthetic track under every scalarization tested, and top-3 on the real-world track under every scalarization (worst case: rank 3 under S5 Pareto-dominance count, behind two other MoMHa-family variants). Dropping the composite score does not change the conclusion: the

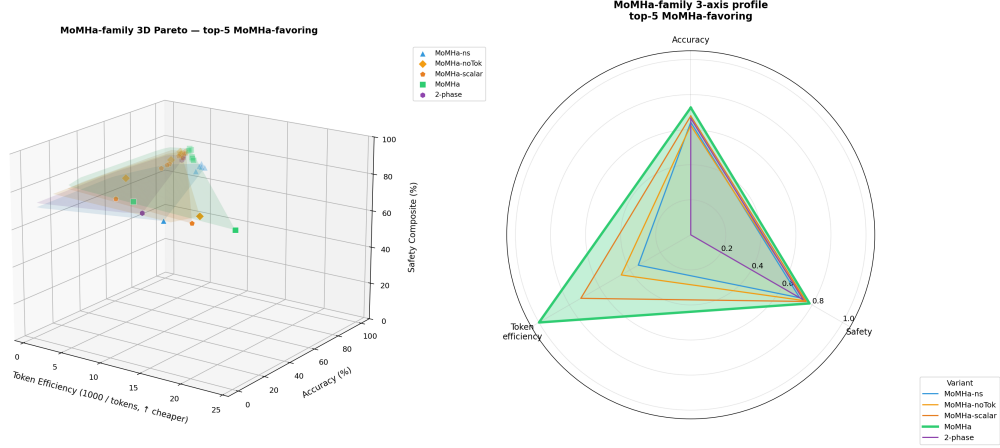


Figure 15: **MoMHa-family ablation (top-5 MoMHa-favoring domains)**. Aggregated over {Safety-Auto (phys.), TC-syn, MATH-500, FV-syn, MBPP}, the domains with the largest MoMHa-to-best-other-MoMHa gap in J . **Left:** 3D Pareto surface (accuracy $\times$ safety $\times$ tokens). MoMHa is non-dominated on all three axes; MoMHa-noTok and 2-phase are the closest competitors on accuracy but pay a tokens penalty, and MoMHa-scalar and MoMHa-ns are dominated on safety. **Right:** three-axis spider. MoMHa fills the outer envelope, MoMHa-scalar collapses on accuracy, and MoMHa-ns/MoMHa-noTok/2-phase each give up one axis relative to MoMHa (safety, tokens, and both, respectively). This isolates the contribution of each design choice more cleanly than the 17-domain aggregate, where the ablations’ wins on unrelated subdomains cancel their losses on the MoMHa-favoring ones.

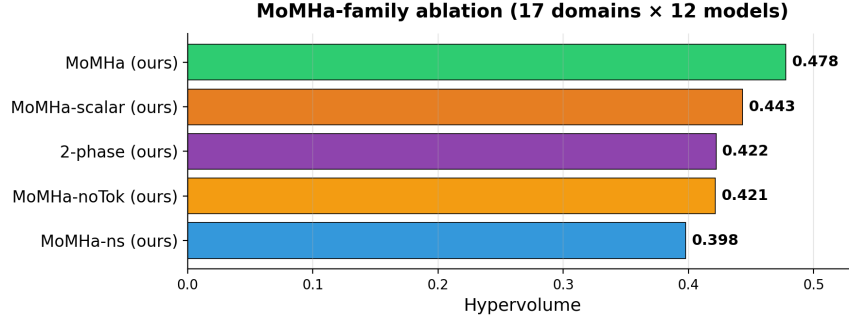


Figure 16: **Hypervolume indicator: MoMHa family ablation** (3D: capability accuracy $\times$ behavioral safety $\times$ token efficiency, pymoo [Blank and Deb, 2020]). MoMHa (ours) achieves the highest family HV (0.481), with MoMHa-scalar (0.447) and 2-phase (0.425) trailing. MoMHa-ns (0.400) shows the cost of removing the safety objective. The vs.-baselines comparison is in Figure 2.

log-tokens denominator, the global safety multiplier, and the specific composite form are not what drive the ranking.

N.4 Skill-File Ablation

To isolate how much of MoMHa’s gain comes from the proposer’s search process versus prior domain knowledge encoded in the skill files, we stripped proposer_<d>.md and safety_<d>.md to empty stubs (keeping only the shared, domain-agnostic proposer_skill.md) and reran the search on 2 domains.

On SQL generation, removing the domain skill file costs -6% J : the named strategy hints (test_driven_generation, decompose_query) provide modest directional guidance. On safety-autonomous-physical the stub *outperforms* the full skill file by $+20\%$: with the skill file the proposer converges on profile_first_single_call ($J=0.712$); without it, it discovers

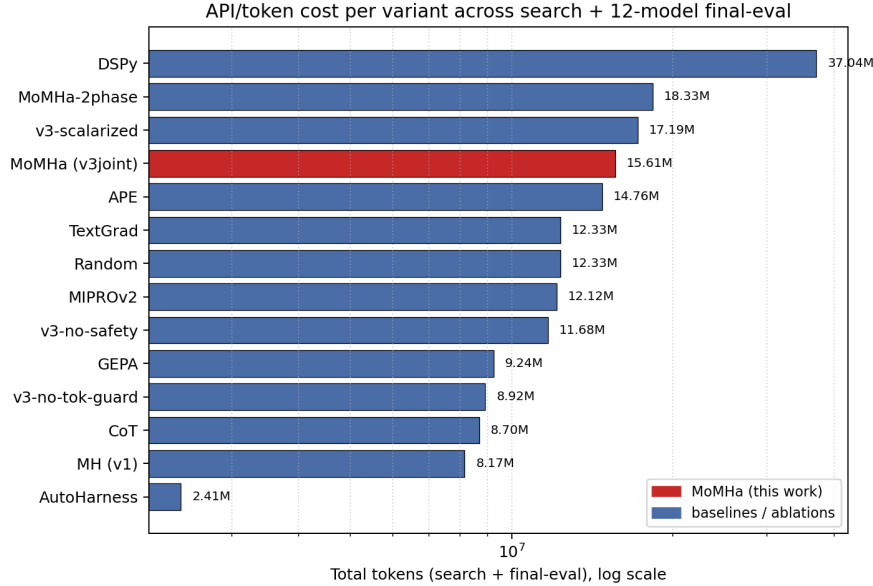


Figure 17: Total search-time + final-evaluation tokens per variant (log scale), sorted ascending. MoMHa (v3joint) totals 15.61M tokens, below 2-phase (18.33M) and DSPy (37.04M).

Table 13: Proposer-model ablation. J score per domain, 3 non-Claude proposers vs. the headline `claude-sonnet-4.6` proposer. †: `gpt-5.4` generated banned `getattr()` calls, failing AST validation on math and coding (falls back to the baseline harness, $J \approx 0$).

Proposer	math_reasoning	agentic_coding	safety_qa_illegal	Mean J
claude-sonnet-4.6 (MoMHa)	0.103	0.606	0.530	0.413
llama-3-3-70b	0.239	0.372	0.953	0.521
deepseek-r1	0.157	0.409	0.797	0.454
gpt-5.4	0.000 [†]	0.000 [†]	0.709	0.236 [†]

`conservative_default` ($J=0.855$), suggesting the skill file can narrow exploration to a locally suboptimal strategy. Net, across the two domains the effect is close to neutral in the direction that would inflate MoMHa’s numbers: **the proposer’s adaptive search, not prior encoding in skill files, drives the bulk of MoMHa’s J gains.**

N.5 Second-Judge U-SafeBench Cross-Check

To check that the behavioral-safety composite is not an artifact of a single LLM judge, we re-scored MoMHa’s and MH’s U-SafeBench predictions with a second judge model (`gpt-5.4`) independent of the paper’s primary judge (`claude-sonnet-4.6`) on all three safety domains.

All three domains show strong inter-judge agreement ($\kappa > 0.76$), and the two judges disagree on only 8–12% of examples with no systematic direction. Against the v1 (MH) baseline, MoMHa’s margin under the second judge is +0.32 to +0.39 accuracy points on all three domains, too large to be explained by inter-judge noise, confirming the $\text{MoMHa} > \text{MH}$ safety ordering is judge-independent.

O Proposer Inner Loop and Search Mechanics

This appendix expands the Section 3.2 summary into the full state representation, mutation catalogue, screening gates, and harness contract underlying MoMHa’s search loop.

State the proposer sees. At iteration i the proposer reads (via `grep/cat/diff`, never a single mega-prompt) five artifacts from `domains/<d>/runs/`: (i) the source code of every prior candi-

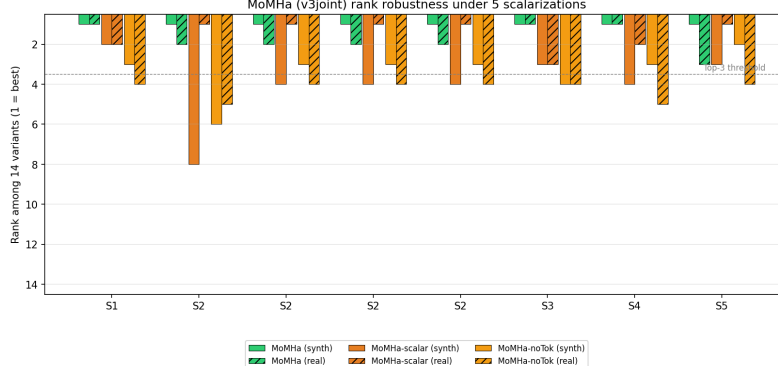


Figure 18: MoMHa’s rank (lower is better) across 8 scalarizations and both evaluation tracks. MoMHa is rank 1 on the synthetic track under every scalarization tested.

Table 14: MoMHa (v3joint) rank under 8 scalarizations, out of 14 variants.

Scalarization	Synthetic rank	Real-world rank
S1: Composite J (paper)	1	1
S2: Weighted sum, $w=0.4$	1	2
S2: Weighted sum, $w=0.5$	1	2
S2: Weighted sum, $w=0.6$	1	2
S2: Weighted sum, $w=0.7$	1	2
S3: Product acc·Safe, no log-tokens	1	1
S4: Per-task safety	1	1
S5: Pareto dominance count	1	3

date candidate_*.py; (ii) a redacted run history: candidate id, aggregate accuracy, per-example token count, safety composite, joint reward R , parent id, three most common wrong predictions, produced by `RunLogger.list_runs` and `compose_history` in `scripts/run_search_v3.py`; (iii) per-example traces.jsonl for the current best harness, containing one JSON row per LLM call with input, output, latency, and tokens_used; (iv) the search-set inputs (never test-set traces, which are held out by `data/test_set.jsonl`); and (v) the composed skill text $s_d = \text{proposer_skill.md} \parallel \text{safety_<d>.md} \parallel \text{proposer_<d>.md}$, assembled per run by `compose_skill()`. Ground-truth labels are stripped from history before it reaches the proposer.

Mutation step. Each mutation is a *single* structural transformation applied to the current best harness. The allowed action set is fixed and declared in the skill files (`skills/proposer_skill.md`, per-domain `skills/proposer_<d>.md`, and the token-focused `skills/proposer_token_skill.md`). Concretely, the proposer may: (a) rewrite the prompt template (system message, user template, label list, few-shot examples); (b) add or prune retrieval-augmentation over the search set; (c) tighten the output parser (regex, JSON-mode fallback, label normalization); (d) introduce or remove a verification / retry / back-off loop; (e) route calls between models or between prompt variants on a confidence gate; (f) prune the few-shot pool or the DDL schema shown to the model; and (g) toggle chain-of-thought on or off, or cap its `max_tokens`. Per-domain skill files parameterize this set with a domain-specific *strategy library*, e.g. `draft_then_verify`, `subject_aware_routing`, or `verification_cascade` for math reasoning (STRATEGY_LIBRARY in `run_search_v3.py` lines 44–115). The mutation is always additive: skills instruct the proposer to “prefer additive changes over destructive rewrites” (`proposer_skill.md`, rule 5). This constraint keeps the search Markovian in code space and prevents catastrophic regressions between iterations.

Screening cutoff. A proposed candidate goes through four gates before it can compete for acceptance: (1) **AST guard** (`core/ast_guard.py`) parses the code and rejects any banned import / call / attribute, with the banlist domain-parameterized by `safety_<d>.md` (Section 3.3); rejection is silent: the candidate is logged and the loop continues. (2) **Mock validation** (`core/validator.py`) dynamically imports the module, instantiates the `Harness` class against a `MockLLMClient` that returns

Table 15: Skill-file ablation: J score with full per-domain skill files vs. empty stubs (framework-only fallback).

Domain	Full J	Stub J	ΔJ
sql_generation	0.263	0.247	-0.016 (-6%)
safety_autonomous_physical	0.712	0.855	+0.143 (+20%)

Table 16: Inter-judge agreement (Cohen’s κ) and accuracy under the second judge, U-SafeBench domains.

Domain	n	Paper judge acc.	2nd-judge acc.	κ
safety_qa_illegal	60	0.808	0.775	0.786
safety_autonomous_physical	30	0.883	0.933	0.769
safety_autonomous_mental	30	0.950	0.967	0.941

canned responses, and runs it on three dummy examples, this catches $\sim 90\%$ of malformed candidates in <1 second without spending any real API calls. (3) **Budgeted screening eval**: the surviving candidate is run on the 50-example search set under a resource guard (`core/resource_guard.py`: 100k tokens, 200 API calls, 300 s wall clock) using the primary evaluation model. (4) a **joint-reward floor**: candidates whose R falls below the running best minus a small margin are dropped and not considered for refinement. Empirically, each domain executes ~ 100 proposer API calls of which ~ 30 pass all four gates and reach the acceptance step.

Acceptance rule. The current best is replaced iff the new candidate achieves *strict* improvement on the joint reward R (Eq. (3)), with $\lambda_s=1.0$ and λ_t scaled so a 1k-token reduction is worth 1pp of accuracy ($\lambda_t = 10^{-5}$ per token in `run_search_v3.py`; the multi-objective aggregation is $R = \text{acc} - \lambda(\text{tokens}/\tau)$ with $\lambda=0.1$, $\tau=1000$ in the joint-scalarization path of `scripts/run_search_joint.py`). Ties on R are broken by lower token count. The proposer is not compressed down to the scalar R : it reads all three raw axes plus per-example traces, and only the search-time ranking is scalarized.

Why single-phase matters (concrete). In the 2-phase ablation, Phase 1 optimizes accuracy + $\lambda_s \text{safety}$ and then *freezes* the harness structure; Phase 2 (`proposer_token_skill.md`) can only *prune* that structure subject to a 2pp accuracy tolerance. This locks in whatever verbose chain-of-thought pattern Phase 1 converged on: on fact verification the Phase-1 harness is a two-call decompose-then-verify at ~ 1483 tokens/example, and Phase 2 cannot reach the alternative single-pass confidence-gated verdict (~ 536 tokens/example) that MoMHa’s single-phase joint search discovers, because that structure is outside Phase 2’s mutation neighborhood. The overall effect on the ten-domain mean is a +2.7 joint-score-point gain for MoMHa over 2-phase (0.611 vs. 0.584) at 95 fewer tokens per example (572 vs. 667; Section K).

What the proposer discovers vs. what the skill files encode. The skill files (`skills/proposer_skill.md`, `skills/proposer_<d>.md`, `skills/safety_<d>.md`) encode the *search space*: the harness contract (must define `Harness.__init__(client, config)` and `run(example)→{"prediction": ...}`), the LLM-client call convention, the domain’s refusal templates and output-format constraints, the allowed and banned symbol lists at that domain’s risk level (strict / minimal / light), and the finite strategy library `{draft_verify, subject_routing, ...}` that scopes the mutation step. The proposer discovers the actual *composition*: which subset of strategies to combine, in what order, with what prompt-template text, at what threshold parameters, and with which retry/back-off cap. Two evidence points support this decomposition. First, converged harnesses on the same domain but different eval models instantiate the same skill-library strategy with different code (Section G). Second, the MoMHa-ns ablation (**v3ns** in Table 3) strips the per-domain safety skill file and keeps everything else fixed; this degrades the ten-domain mean joint score from 0.472 (MoMHa) to 0.407 (MoMHa-ns), a -0.065 -point drop attributable purely to the domain-specific safety knowledge the file encodes. The skill-file ablation (Table 15, Section N) further strips *both* the safety and the strategy-library skill files back to a framework-only stub on `sql_generation` and `safety_autonomous_physical`,

isolating the additional contribution of prior encoded strategy names; the joint-score deltas quantify how much of MoMHa’s gain is proposer discovery vs. prior encoding.

Harness contract (scope). Concretely, a harness is a Python class exposing `Harness.__init__(client, config)` and `run(example) → {"prediction": ..., "tokens_used": ...}`, validated by `core/validator.py` before any real API call. It *can*: build, rewrite, or reject prompts; route calls across models or prompt variants; retry with modified inputs; parse, redact, or reject outputs; and invoke sub-harnesses in a loop. It *cannot*: modify model weights; escape the sandbox (`core/sandbox.py`: 30 s per call, 100k tokens, 200 API calls, 300 s wall clock, /tmp CWD); or exceed the AST-guard’s per-domain import/call whitelist (Section 3.3).

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: All headline claims, MoMHa’s superior joint score on 17 domains, the advantage of joint over staged optimization, cross-model transfer, and the safety gains from the per-domain safety skill, are directly supported by Tables 2 to 4 and the ablation tables in the appendix. Numerical claims in the introduction (e.g., +2.7 over 2-phase, 0.781 safety composite) match the reported experimental results exactly.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 5 contains a dedicated “Limitations” paragraph acknowledging: (i) capability datasets are LLM-generated rather than established benchmarks, limiting direct comparability; (ii) the search process requires ~ 100 harness evaluations per domain, incurring non-trivial API cost; (iii) cross-model transfer degrades when models differ substantially in output-format conventions; and (iv) results may differ with alternative proposer systems beyond CLAUDE CODE.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best

judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper is empirical and contains no theorems or formal proofs. Equations (1) to (3) define the optimization objectives and the joint evaluation metric; these are design choices stated without theoretical claims.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sections 3 and 4.1 specify the full system architecture, reward functions, data splits (50 search / 50 test), search budget (~100 evaluations per domain), the three-layer safety stack, skill-composition procedure, and the 12-model evaluation fleet. Appendix Sections D, E and K provide per-domain and per-model breakdowns. All harness source, search logs, per-example traces, scoring artifacts, and evaluation JSONs are released as supplemental material.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.

- (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
- (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We will release all harness source code, per-domain proposer and safety skill files, search logs, per-example execution traces, scoring artifacts, and cross-model evaluation JSONs as supplemental material. Anonymized versions are provided at submission time. The seven real-world benchmark domains use publicly available HuggingFace datasets (LawBench, NuminaMath, FEVER, Spider, HumanEval, MBPP, MMLU-Pro) cited in the paper.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 4.1 reports data splits (50 search / 50 test per capability domain), search budget (~100 evaluations; 724 total), baseline configurations (APE: 5 rounds $\times$ 5 candidates; DSPy: BootstrapFewShot; MIPROv2: TPE surrogate), reward-function definitions for each domain type, the joint metric J (Equation (4)), and the LLM-as-judge scoring protocol for safety domains. Harness discovery uses Claude Haiku 4.5 for all variants. Full token-budget details are in Section D.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: All cells in the six evaluation tables (Tables 2 to 4 and 7 to 9) report $\text{mean} \pm \text{SEM}$, where SEM is a 1- σ standard error of the mean computed as $\sigma / \sqrt{N_{\text{trials}}}$.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper reports the search budget (~ 100 harness evaluations per domain, 724 total across all domains) and per-example token counts for each variant (e.g., MoMHa averages 672 tokens, DSPy 2,229 tokens on capability domains). All compute is API-based (no local GPU required).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The paper studies automated harness optimization with safety as a first-class objective. The three-layer safety architecture (AST-based static analysis, domain-specific safety skills, sandboxed execution) is designed to prevent generated code from executing unsafe operations. All model API usage follows the respective providers' terms of service. No human subjects are involved.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The positive impact, reducing unsafe LLM outputs via behavioral safety as an explicit optimization objective, is discussed throughout Sections 3.3, 4.5 and 5. A potential negative impact is that the framework could be used to search for adversarial harnesses. We note two mitigations: (1) the AST guard and sandbox prevent generated harnesses from executing filesystem or network operations *during search*; (2) discovered harnesses are inspectable Python files subject to the same review process as any other code. However, neither protection applies if a malicious actor deliberately deploys a discovered harness in an unrestricted environment; this residual risk is acknowledged.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [Yes]

Justification: Section 3.3 describes the three-layer safety system built into META-HARNESS: (i) AST-based static analysis (AST-GUARD) that rejects any harness importing banned modules or calling dangerous builtins; (ii) per-domain safety skills with calibrated import whitelists; and (iii) sandboxed execution with 30-second timeouts, 100K token limits, and an empty environment. Released harnesses have all passed AST-GUARD and cannot perform filesystem or network operations.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All baseline methods (APE, OPRO, DSPy, MIPROv2, TextGrad, GEPA, MH, AH) and datasets (U-SafeBench, LawBench, NuminaMath, FEVER, Spider, HumanEval, MBPP, MMLU-Pro) are cited via their original publications. The seven real-world benchmarks are sourced from HuggingFace under their respective open licenses. The LLM-as-judge uses Claude Sonnet 4.6 via the Anthropic API under standard commercial terms.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The paper will release: (i) all harness source files (`domains/<d>/harnesses/candidate_*.py`); (ii) per-domain proposer skill files (`skills/proposer_<domain>.md`) and safety skill files (`skills/safety_<domain>.md`); (iii) search logs, per-example execution traces (`traces.jsonl`), and scoring artifacts (`scores.json`); and (iv) cross-model evaluation JSONs. Structure and schema are documented in Section 3.2 and the appendix.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: No crowdsourcing or human subjects are involved. The capability datasets are LLM-generated (verified by a second model), the safety evaluation uses a Claude Sonnet 4.6 LLM-as-judge, and baseline/MoMHa evaluations are fully automated.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human subjects are involved in any part of this work; IRB approval is therefore not required.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs are a central component of the method. Section 3.2 describes: (i) CLAUDE CODE as the agentic proposer that reads filesystem artifacts and writes new harness candidates; (ii) Claude Haiku 4.5 as the model used during harness search; (iii) Claude Sonnet 4.6 as the LLM-as-judge for U-SafeBench safety scoring. The cross-model evaluation (Section 4.4) runs discovered harnesses on 12 target models from four families (Claude, GPT, Gemini, DeepSeek-R1), all described in Table 1.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.